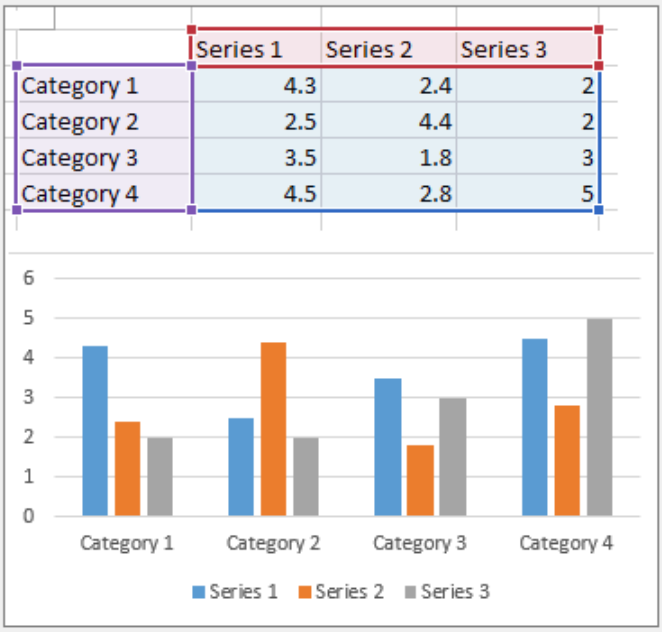
Introduction to Matplotlib

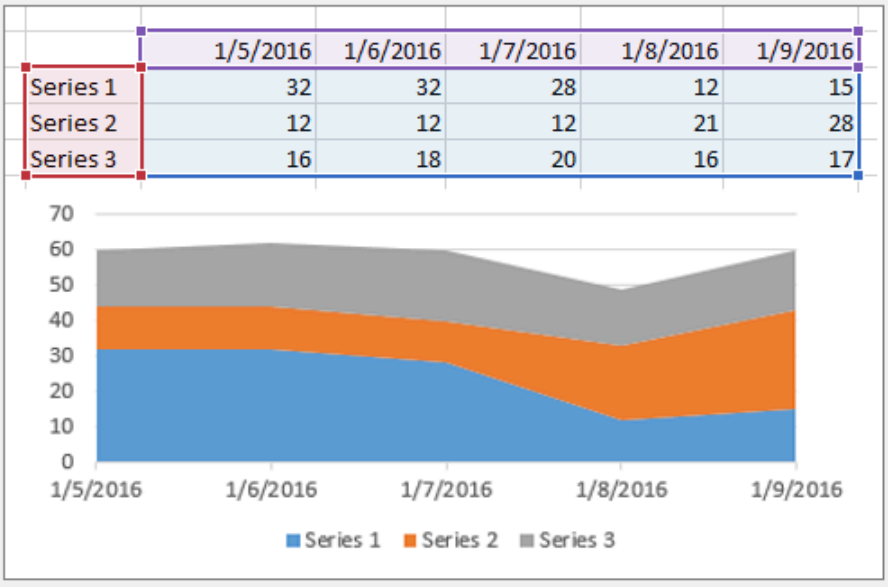
In [ ]:
```
Introduction to Matplotlib
- 10.1 Basic Methods -
-     Plot()
-     Markers
-     colors
-     linestyles
-     show()
- Describing the plot -
- • xlabel ylabel, title with different fonts, colors and positions
- • Grids
- • Subplots
- 10.2
- • Simple Scatterplots with color, size and alpha
- • Simple Bar charts bar() and barh()
- • Simple Histogram with hist() with only x, bins, orientation, align attributes
- • Simple pie charts with pie along with labelling and coloring
```

Why Data Visulization

- Data visualization is quick,easy way to convey concept in a universal manner



	Series 1	Series 2	Series 3
Category 1	4.3	2.4	2
Category 2	2.5	4.4	2
Category 3	3.5	1.8	3
Category 4	4.5	2.8	5



	1/5/2016	1/6/2016	1/7/2016	1/8/2016	1/9/2016
Series 1	32	32	28	12	15
Series 2	12	12	12	21	28
Series 3	16	18	20	16	17

-

Vishal Acharya

What is Data Visualization

- It is the graphical representation of information and data.
- visual elements like charts, graphs, and maps, data visulizatiob tools provide an accessible way to see and understand trends, outliers and present in data

Type of data visualization in python

- matplotlib: low level,provide lots of freeedom
- pandas visualization: easy to use interface,built in Matplotlib
- seaborn: high-level interface, great default style
- ggplot:based on R's ggplot2,uses grammar of graphics
- plotpy: can create interactive plots

Data visualisation

- Data visualisation means graphical or pictorial representation of the data using graph, chart, etc.
- The purpose of plotting data is to visualise variation or show relationships between variables.
- Visualisation also helps to effectively communicate information to intended users. Traffic symbols,ultrasound reports, Atlas book of maps, speedometer of a vehicle, tuners of instruments are few examples of visualisation that we come across in our daily lives. Visualisation of data is effectively used in fields like health, finance, science, mathematics, engineering, etc. In this chapter, we will learn how to visualise data using Matplotlib library of Python by plotting charts such as line, bar, scatter, histrogram, pie with respect to the various types of data.

What is matplotlib?

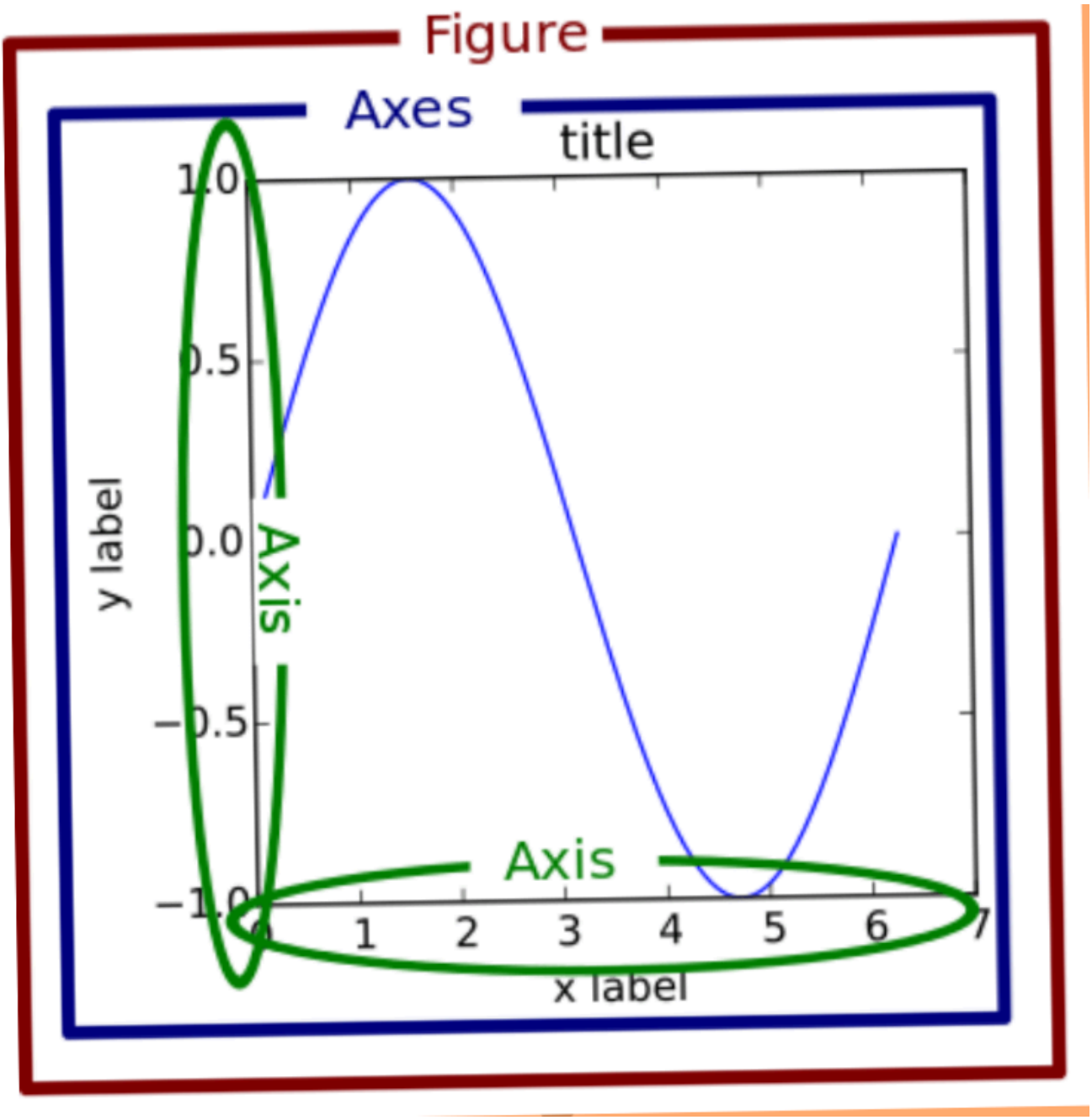
- matplotlib is a plotting library for the python programming language and its numerical mathematics extension Numpy
- it is a python library used for data visualization
- it is 2d and 3d plotting python livbrary
- it was introduced by jhon hunter in year 2002

Matplotlib Graphs

- Linear plot: It displays a sequence of data points as markers. The points are ordered typically by their x-axis value. These points are joined with straight line segments. A line graph is used to visualize a trend in data over intervals of time.
- scatter plot: It is a type of plot using Cartesian coordinates to display values for two variables for a set of data. It is displayed as a collection of points. Their position on the horizontal axis determines the value of one variable. The position on the vertical axis determines the value of the other variable. A scatter plot can be used when one variable can be controlled and the other variable depends on it. It can also be used when both continuous variables are independent.
- bar plot: A bar graph is a graph that presents categorical data with rectangle-shaped bars. The heights or lengths of these bars are proportional to the values that they represent. The bars can be vertical or horizontal. A vertical bar graph is sometimes called a column graph.
- stem plot:A stem plot is quite similar to a bar chart. The difference is that, instead of using a bar for each data point, a stem plot uses a vertical line with a marker at the top.

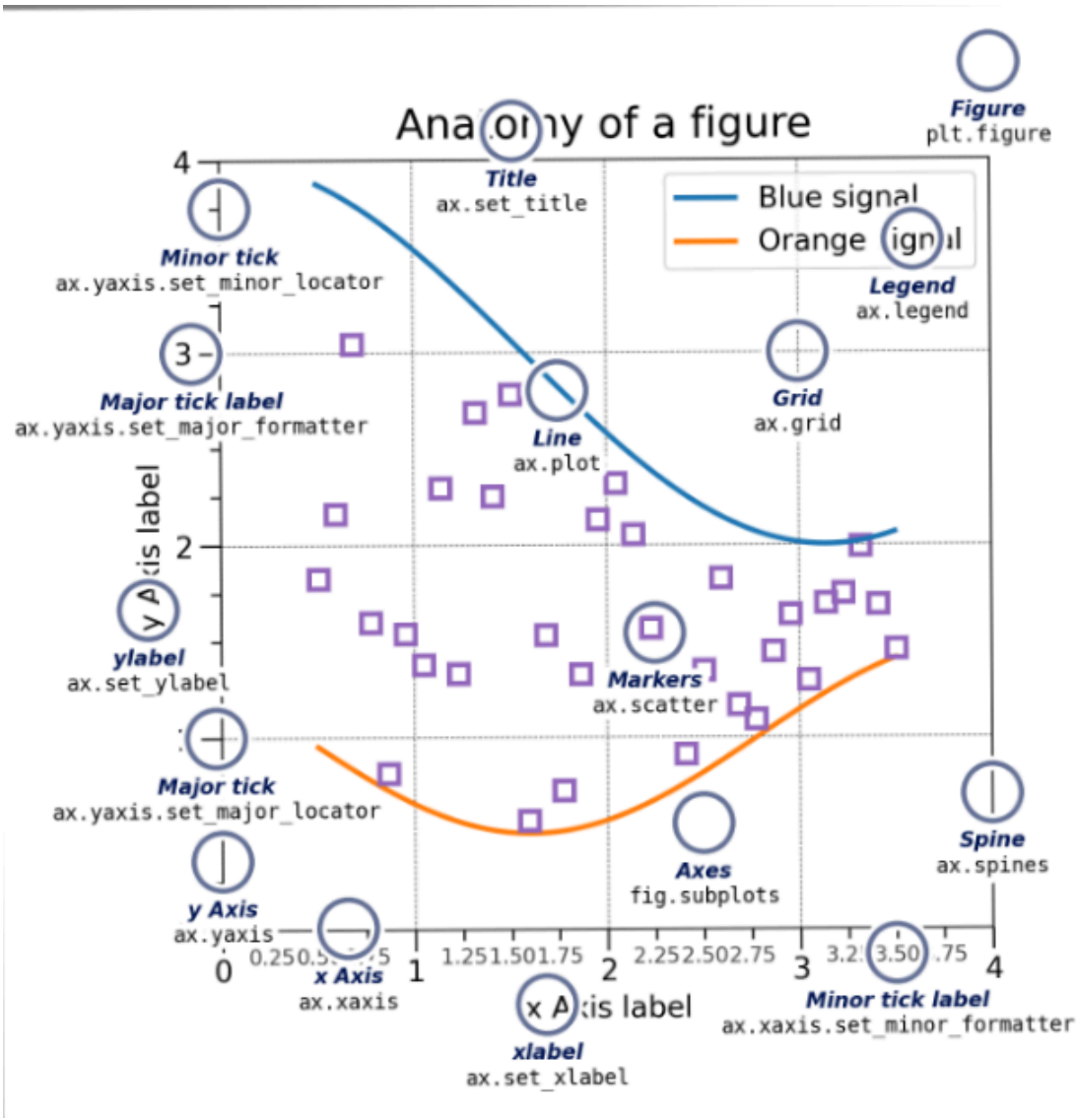
Vishal Acharya

- hist plot: A histogram is an approximate representation of the distribution of numerical data. The data is divided into non-overlapping intervals called bins and buckets. A rectangle is erected over a bin whose height is proportional to the number of data points in the bin. Histograms give a feel of the density of the distribution of the underlying data.
- box plot: Box Plot is a good way of looking at how data is distributed. It has a box as the name suggests. One end of the box is at the 25th percentile of the data. 25th percentile is the line drawn where 25% of the data points lie below it. The other end of the box is at the 75th percentile (which is defined similarly to the 25th percentile as above). The median of the data is marked by a line. There are two additional lines which are called whiskers. The 25th percentile mark is termed 'Q1' (representing the first quarter of the data). 75th percentile is Q3. The difference between Q3 and Q1 (Q3 – Q1) is IQR (Inter Quartile Range). Whiskers are marked at last data points on either side within the extreme range of Q1 – 1.5 * IQR and Q3 + 1.5 * IQR. The data points outside these whiskers are called 'outliers' as they deviate significantly from the rest of the data points.
- pie plot: A pie chart is a circular statistical graphic. To illustrate numerical proportion, it is divided into slices. In a pie chart, for every slice, each of its arc lengths is proportional to the amount it represents. The central angles, and area are also proportional. It is named after a sliced pie.
- Fill_Between_plot: is used to fill areas of color between specified values on your plot. This is typically done to highlight or emphasize portions of the plot, or to make an obvious distinction between different portions of the plot. For example, you may want to highlight the area between Trading Bands in a Bollinger plot. Or color a positive moving average area differently from negative moving average area.



Vishal Acharya

Vishal Acharya



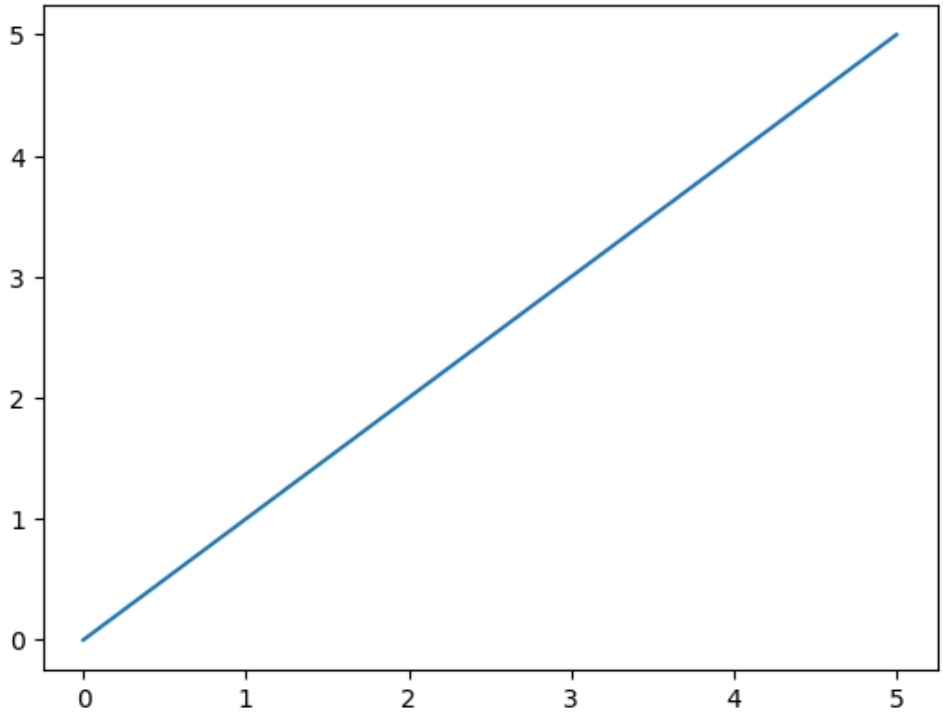
pyplot

- `text()` : adds text in an arbitrary location
- `xlabel()` : adds text to the x-axis
- `ylabel()` : adds text to the y-axis
- `title()` : adds title to the plot
- `clear()` : removes all plots from the axes.
- `savefig()` : saves your figure to a file
- `legend()` : shows a legend on the plot

All methods are available on `pyplot` and on the axes instance generally.

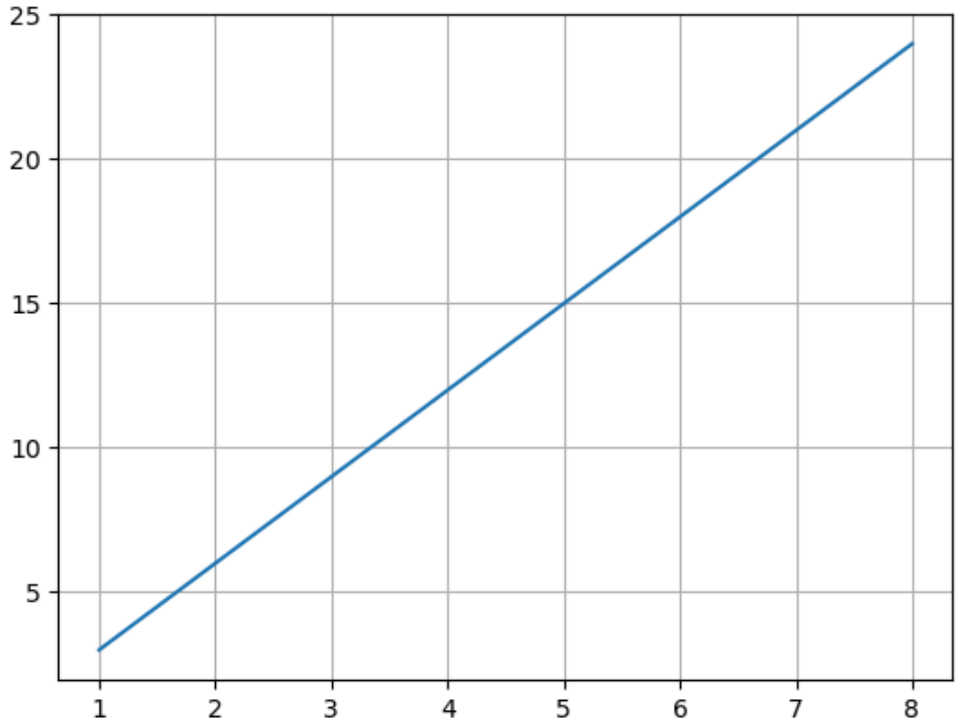
In [1]:
```
#simple line plot
import matplotlib.pyplot as plt
x1=[0,1,2,3,4,5]
y1=[0,1,2,3,4,5]
```

```
plt.plot(x1,y1)
plt.show()
```



for grid

In [60]:
```
import numpy as np
x=np.array([1,8])
y=np.array([3,24])
plt.plot(x,y)
plt.grid()
plt.show()
```

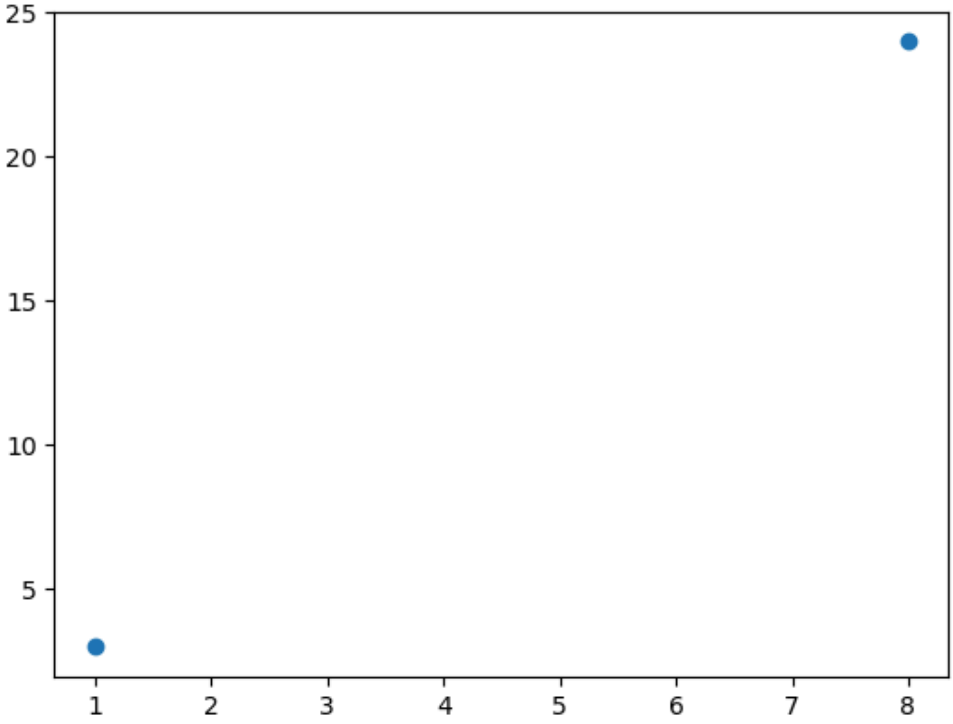


only marker

In [5]:
```
x=np.array([1,8])
y=np.array([3,24])
```

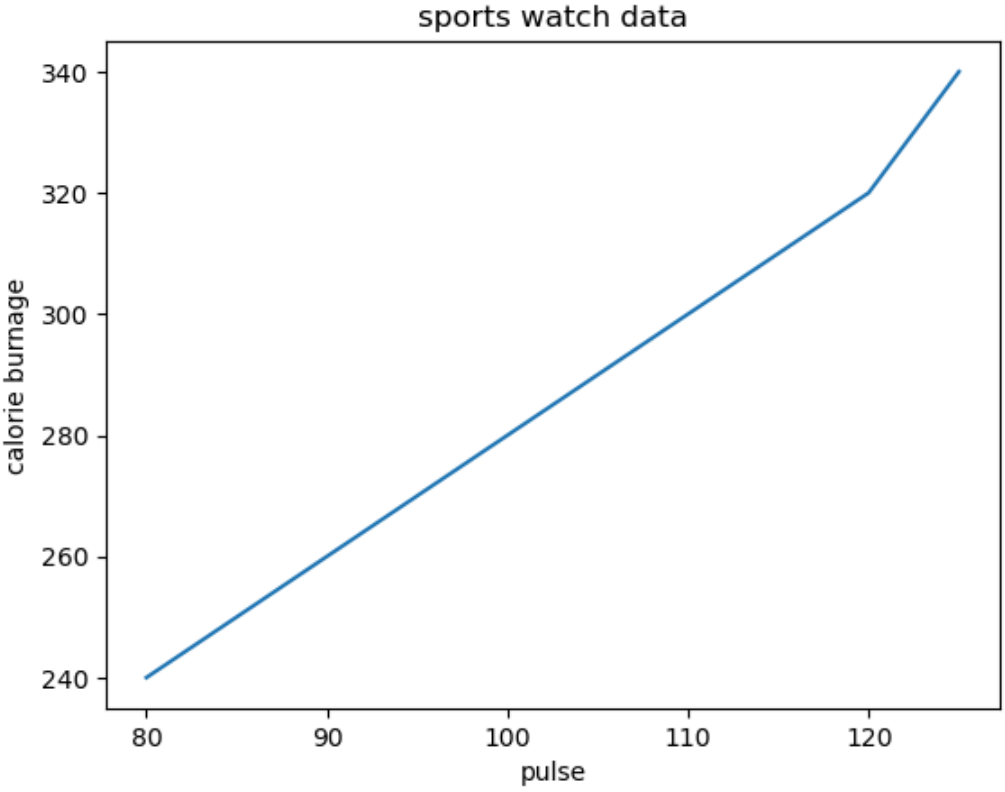
Vishal Acharya

```
plt.plot(x,y,"o")
plt.show()
```



Create labels and title for a plot

```
In [6]: x=np.array([80,85,90,95,100,105,110,115,120,125])
y=np.array([240,250,260,270,280,290,300,310,320,340])
plt.plot(x,y)
plt.xlabel("pulse")
plt.ylabel("calorie burnage")
plt.title("sports watch data")
plt.show()
```



set font properties for title,legend and labels

Vishal Acharya

for legend location

Location string	Location code
Best	0
upper right	1
upper left	2
lower left	3
lower right	4
Right	5
center left	6
Center right	7
lower center	8
upper center	9
Center	10

!

- axes.plot()
- This is the basic method of axes class that plots values of one array versus another as lines or markers. The plot() method can have an optional format string argument to specify color, style and size of line and marker.

Vishal Acharya

Color codes

Character	Color
'b'	Blue
'g'	Green
'r'	Red
'c'	Cyan
'm'	Magenta
'y'	Yellow
'k'	Black
'w'	White

Marker codes

Character	Description
'.'	Point marker
'o'	Circle marker
'x'	X marker
'D'	Diamond marker
'H'	Hexagon marker
's'	Square marker
'+'	Plus marker

Line styles

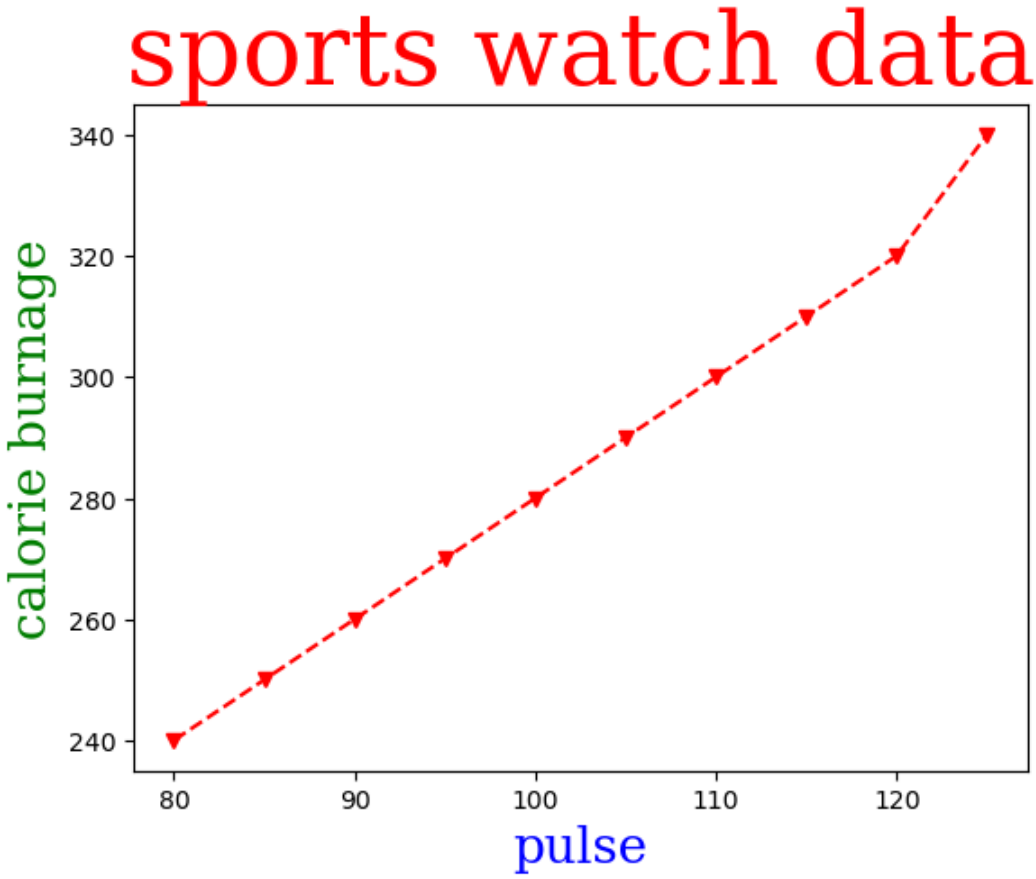
Character	Description
'-'	Solid line

'—'	Dashed line
'-.'	Dash-dot line
':'	Dotted line

ms=10 #marker size

In [32]:
```
x=np.array([80,85,90,95,100,105,110,115,120,125])
y=np.array([240,250,260,270,280,290,300,310,320,340])
plt.plot(x,y,"rv--")
style1={'family':'serif','color':'blue','size':20}
style2={'family':'serif','color':'green','size':20}
style3={'family':'serif','color':'red','size':40}
plt.xlabel("pulse",fontdict=style1)
plt.ylabel("calorie burnage",fontdict=style2)
plt.title("sports watch data",fontdict=style3)
plt.show()
```

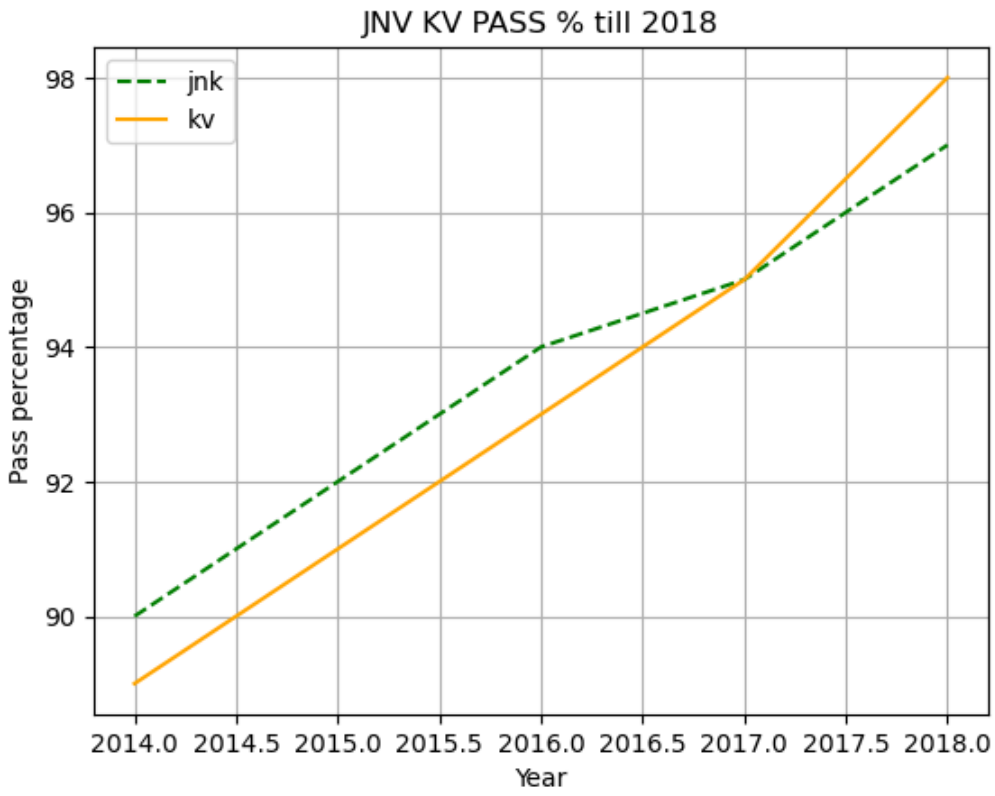
Vishal Acharya



In [29]:
```
x=np.array([80,85,90,95,100,105,110,115,120,125])
y=np.array([240,250,260,270,280,290,300,310,320,340])
plt.plot(x,y,label="legend")
plt.xlabel("pulse",fontsize=30,fontweight="bold",fontfamily="serif",
           color="red")
plt.ylabel("calorie burnage")
plt.title("sports watch data")
plt.legend()
plt.show()
```

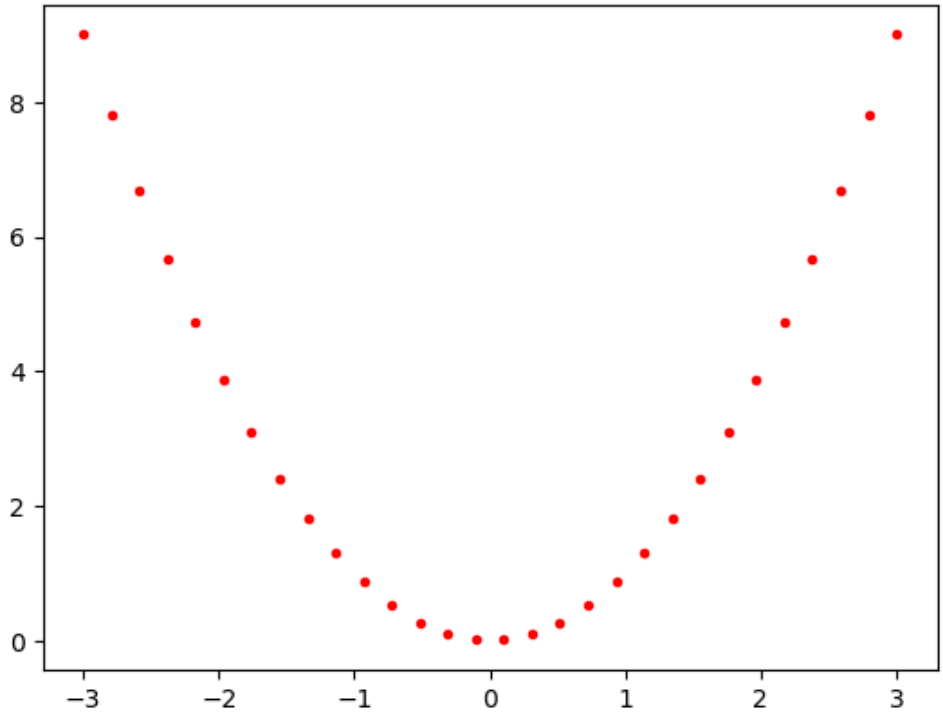
Vishal Acharya

In [59]:
```
import numpy as np
import matplotlib.pyplot as plt
year = [2014,2015,2016,2017,2018]
jnvpasspercentage = [90,92,94,95,97]
kvpasspercentage = [89,91,93,95,98]
plt.plot(year, jnvpasspercentage, color='g',linestyle="--",label="jnk")
plt.plot(year, kvpasspercentage, color='orange',label="kv")
plt.xlabel("Year")
plt.ylabel('Pass percentage')
plt.title('JNV KV PASS % till 2018')
plt.legend()
plt.grid()
plt.show()
```

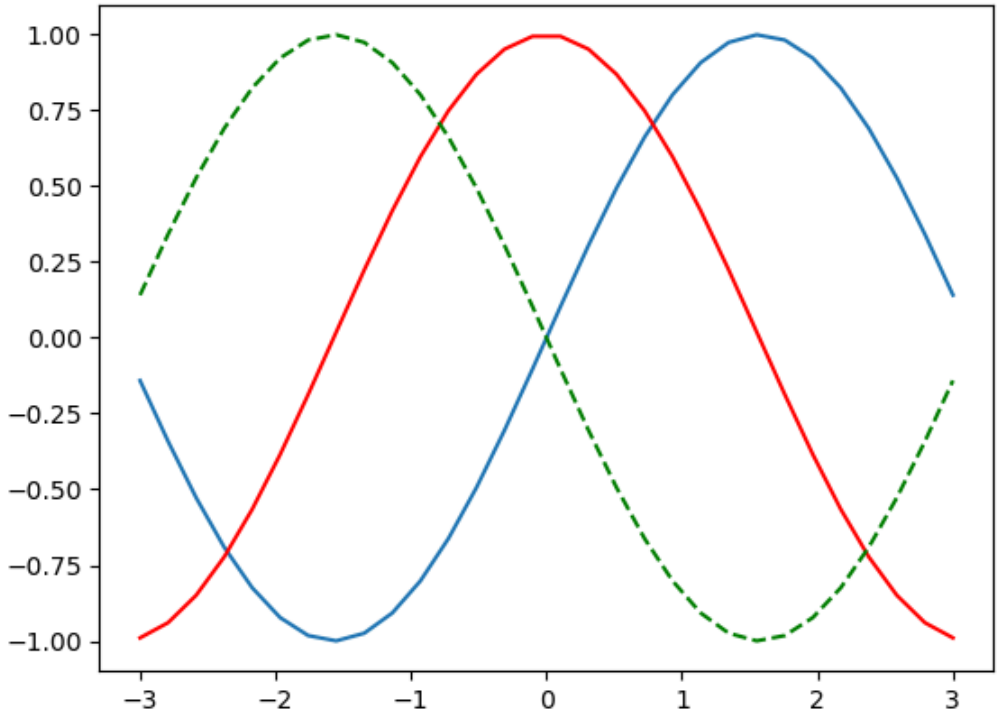


In [22]:
```
x = np.linspace(-3, 3, 30)
y = x**2
plt.plot(x, y, 'r.')
plt.show()
```

Vishal Acharya



In [25]:
```
x = np.linspace(-3, 3, 30)
plt.plot(x, np.sin(x))
plt.plot(x, np.cos(x), 'r-')
plt.plot(x, np.sin(-x), 'g--')
plt.show()
```

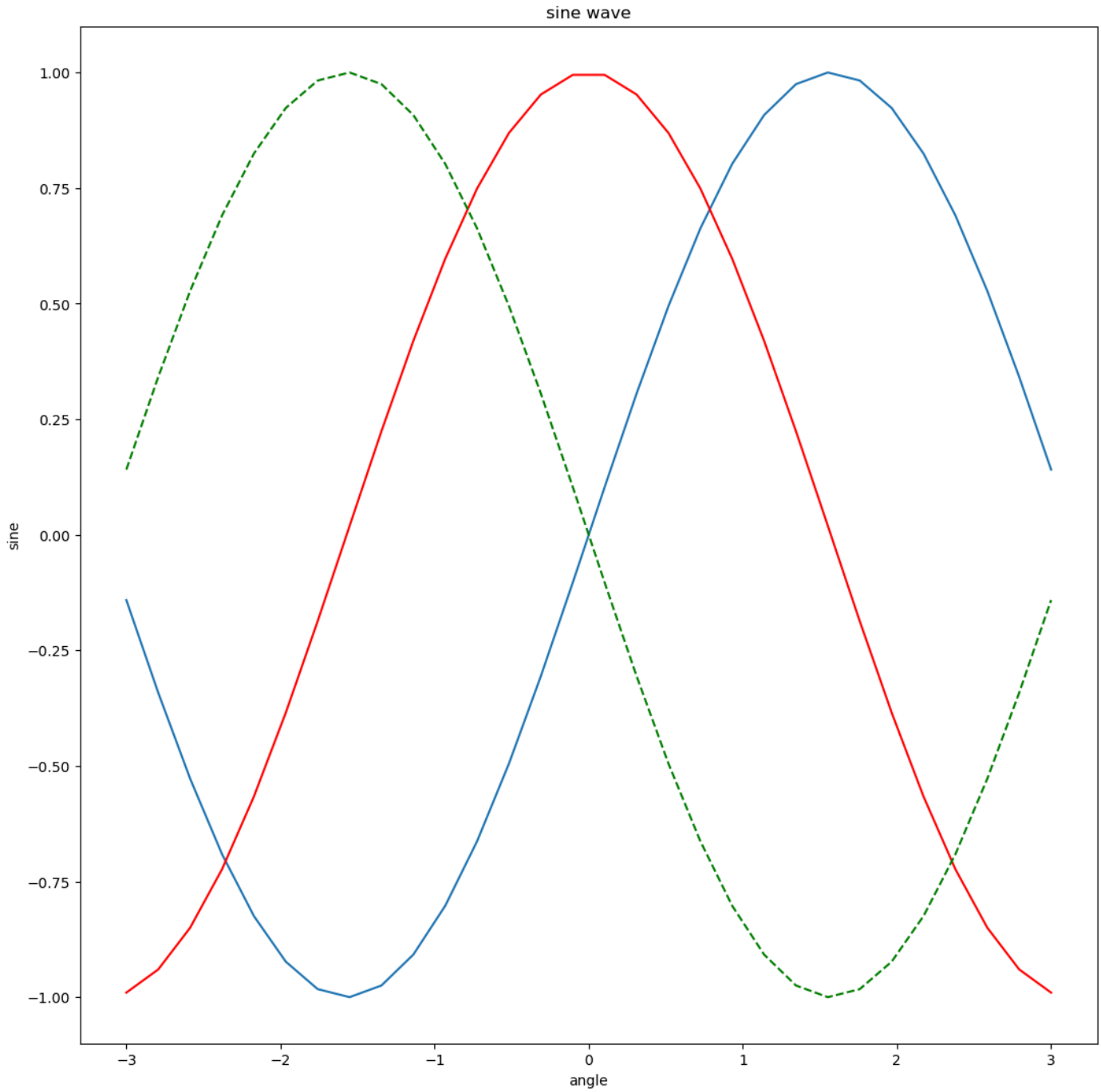


Vishal Acharya

Matplotlib – Object-oriented Interface

- In object-oriented interface, Pyplot is used only for a few functions such as figure creation, and the user explicitly creates and keeps track of the figure and axes objects. At this level, the user uses Pyplot to create figures, and through those figures, one or more axes objects can be created. These axes objects are then used for most plotting actions.
- Now add axes to figure. The add_axes() method requires a list object of 4 elements corresponding to left, bottom, width and height of the figure. Each number must be between 0 and 1:

In [68]:
```
x = np.linspace(-3, 3, 30)
fig = plt.figure(figsize=(10,10))
ax=fig.add_axes([0,0,1,1])
plt.plot(x, np.sin(x))
plt.plot(x, np.cos(x), 'r-')
plt.plot(x, np.sin(-x), 'g--')
ax.set_title("sine wave")
ax.set_xlabel('angle')
ax.set_ylabel('sine')


plt.show()
```

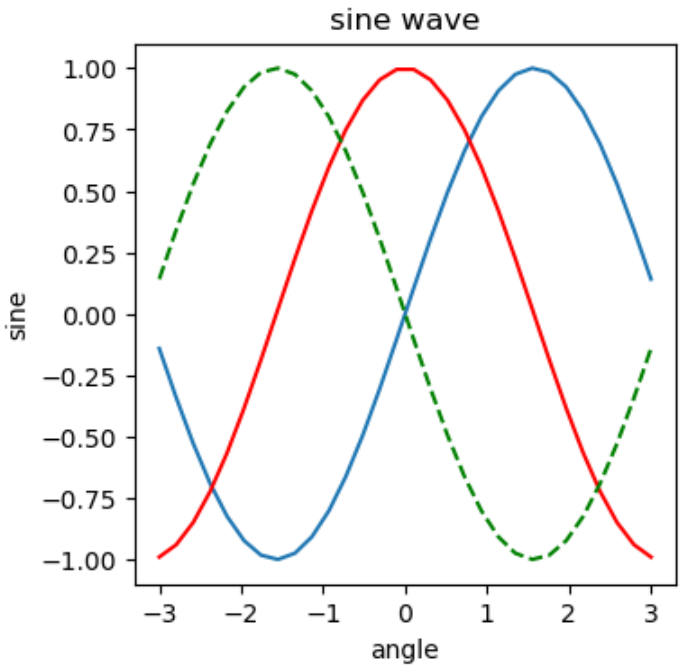


Vishal Acharya

- The matplotlib.figure module contains the Figure class. It is a top-level container for all plot elements. The Figure object is instantiated by calling the figure() function from the pyplot module

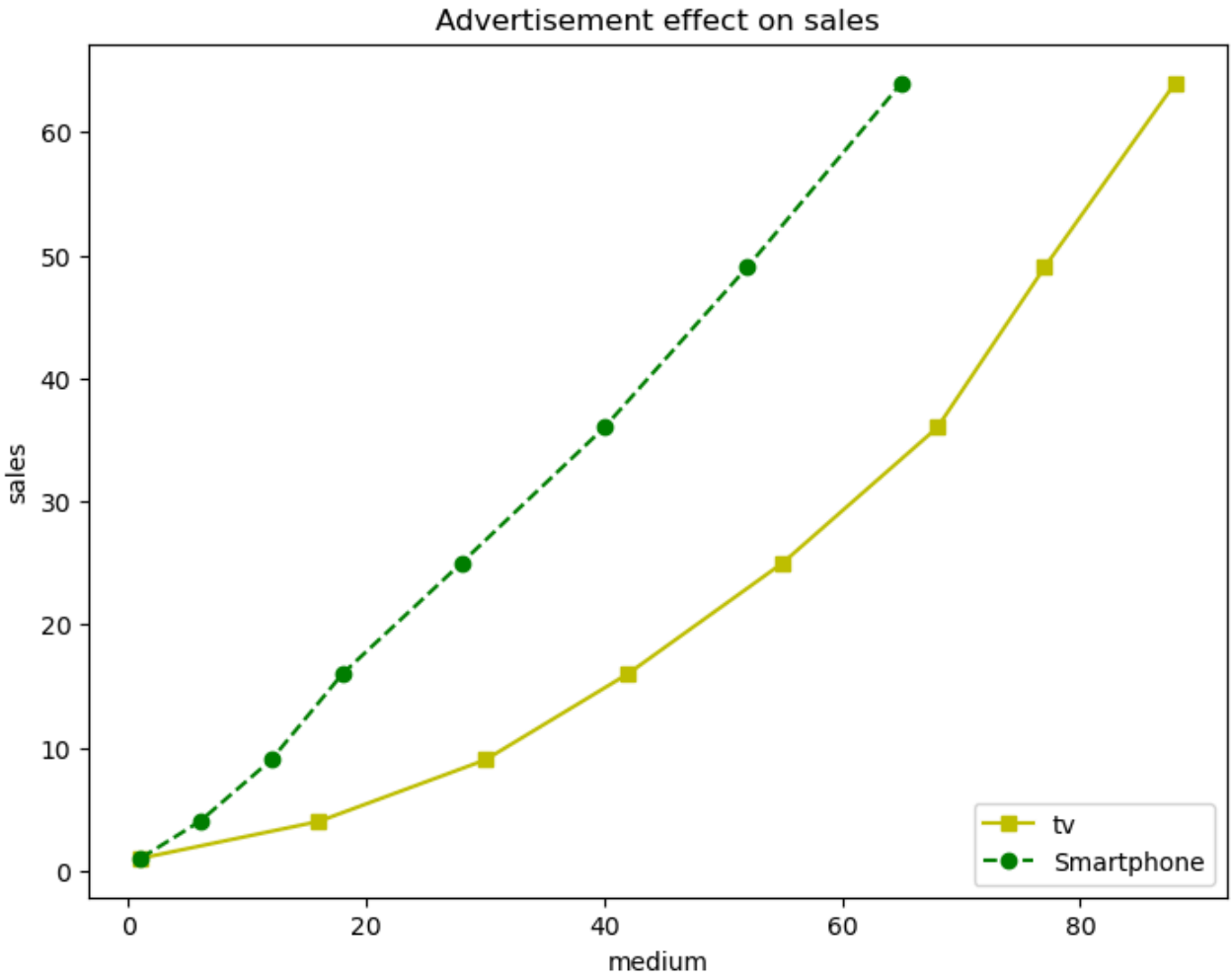
Figsize	(width,height) tuple in inches
Dpi	Dots per inches
Facecolor	Figure patch facecolor
Edgecolor	Figure patch edge color
Linewidth	Edge line width

In [70]:
```
x = np.linspace(-3, 3, 30)
fig = plt.figure(figsize=(3,3))
ax=fig.add_axes([0,0,1,1])
plt.plot(x, np.sin(x))
plt.plot(x, np.cos(x), 'r-')
plt.plot(x, np.sin(-x), 'g--')
ax.set_title("sine wave")
ax.set_xlabel('angle')
ax.set_ylabel('sine')


plt.show()
```



In [34]:
```
import matplotlib.pyplot as plt
y = [1, 4, 9, 16, 25,36,49, 64]
x1 = [1, 16, 30, 42,55, 68, 77,88]
x2 = [1,6,12,18,28, 40, 52, 65]
fig=plt.figure()
ax=fig.add_axes([0,0,1,1])
l1=ax.plot(x1,y,'ys-') # solid line with yellow colour and square marker
l2=ax.plot(x2,y,'go--') # dash line with green colour and circle marker
ax.legend(labels=('tv', 'Smartphone'), loc='lower right') # legend placed at lower right
ax.set_title("Advertisement effect on sales")
ax.set_xlabel('medium')
ax.set_ylabel('sales')
plt.show()
```

Vishal Acharya

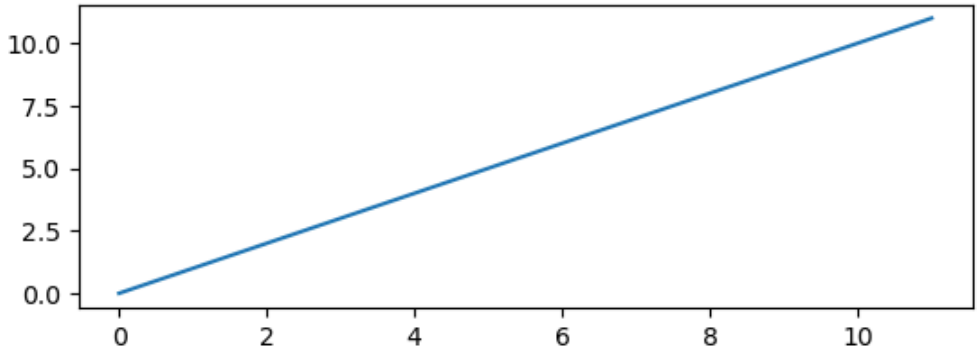


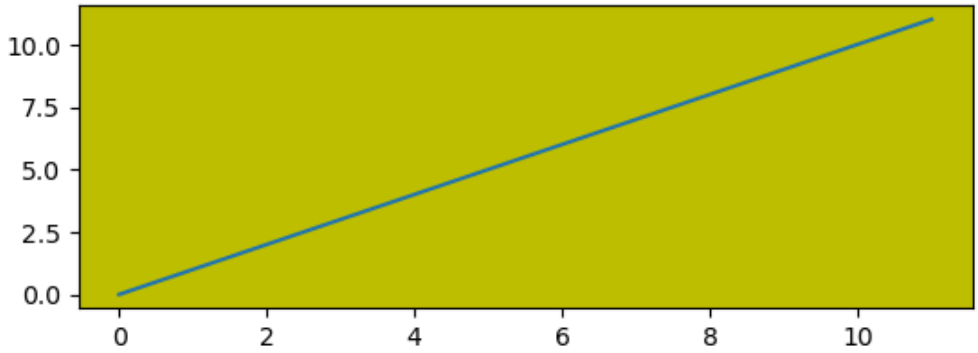
subplot()

- The subplot() function returns the axes object at a given grid position. The Call signature of this function is:
- `plt.subplot(subplot(nrows, ncols, index)`
- In the current figure, the function creates and returns an Axes object, at position index of a grid of nrows by ncolsaxes. Indexes go from 1 to nrows * ncols, incrementing in row-major order.Ifnrows, ncols and index are all less than 10. The indexes can also be given as single, concatenated, threedigitnumber.
- For example, subplot(2, 3, 3) and subplot(233) both create an Axes at the top right corner of the current figure, occupying half of the figure height and a third of the figure width.
- Creating a subplot will delete any pre-existing subplot that overlaps with it beyond sharing a boundary.

In [71]:
```
import matplotlib.pyplot as plt
# plot a line, implicitly creating a subplot(111)
plt.plot([1,2,3])
# now create a subplot which represents the top plot of a grid with 2 rows and 1 column.
#Since this subplot will overlap the first, the plot (and its axes) previously created, will be removed
plt.subplot(211)
plt.plot(range(12))
plt.subplot(212, facecolor='y') # creates 2nd subplot with yellow background
plt.plot(range(12))
plt.show()
```

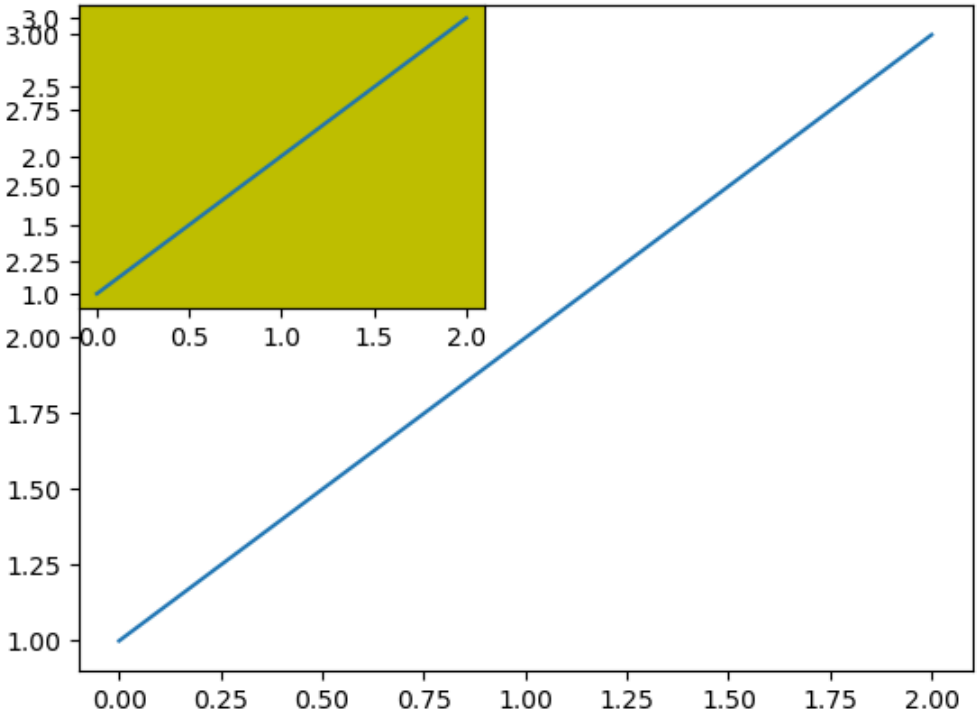
Vishal Acharya





The add_subplot() function of the figure class will not overwrite the existing plot:

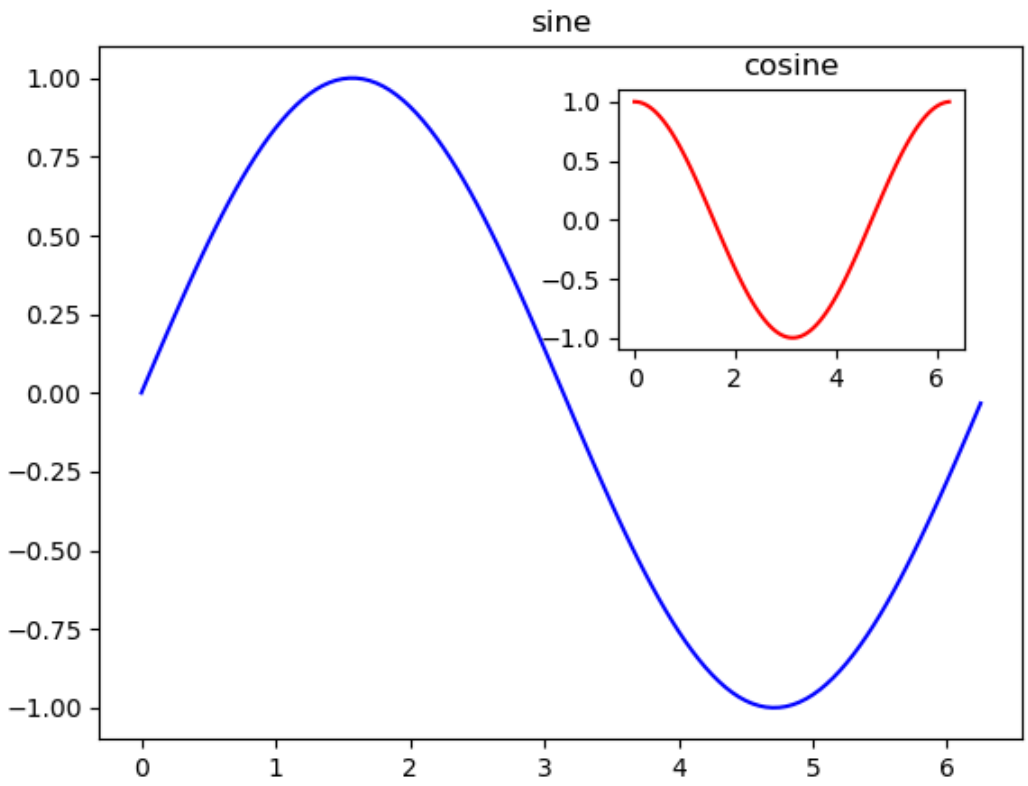
In [37]:
```
import matplotlib.pyplot as plt
fig=plt.figure()
ax1=fig.add_subplot(111)
ax1.plot([1,2,3])
ax2=fig.add_subplot(221, facecolor='y')
ax2.plot([1,2,3])
plt.show()
```



In [39]:
```
import numpy as np
import math
x=np.arange(0, math.pi*2, 0.05)
fig=plt.figure()
axes1 = fig.add_axes([0.1, 0.1, 0.8, 0.8]) # main axes
axes2 = fig.add_axes([0.55, 0.55, 0.3, 0.3]) # inset axes
```

Vishal Acharya

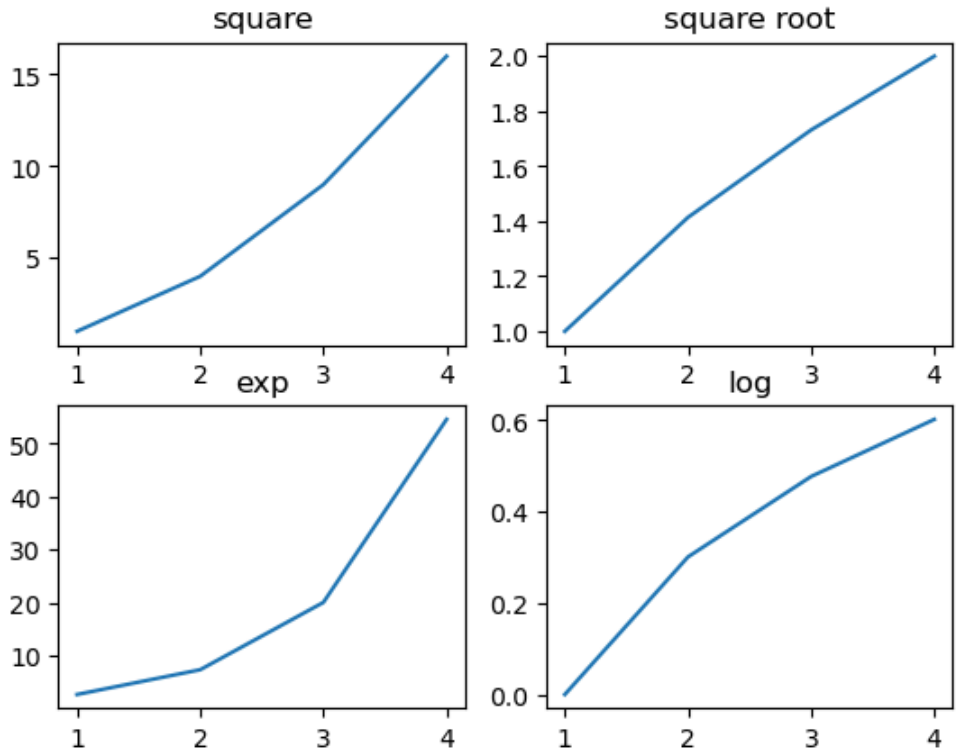
```
y=np.sin(x)
axes1.plot(x, y, 'b')
axes2.plot(x,np.cos(x),'r')
axes1.set_title('sine')
axes2.set_title("cosine")
plt.show()
```



- Matplotlib'spyplot API has a convenience function called subplots() which acts as a utility wrapper and helps in creating common layouts of subplots, including the enclosing figure object, in a single call.
- Plt.subplots(nrows, ncols)
- The two integer arguments to this function specify the number of rows and columns of the subplot grid. The function returns a figure object and a tuple containing axes objects equal to nrows*ncols. Each axes object is accessible by its index. Here we create a subplot of 2 rows by 2 columns and display 4 different plots in each subplot.

In [41]:
```
#rows by 2 columns and display 4 different plots in each subplot.
import matplotlib.pyplot as plt
fig,a= plt.subplots(2,2)
import numpy as np
x=np.arange(1,5)
a[0][0].plot(x,x*x)
a[0][0].set_title('square')
a[0][1].plot(x,np.sqrt(x))
a[0][1].set_title('square root')
a[1][0].plot(x,np.exp(x))
a[1][0].set_title('exp')
a[1][1].plot(x,np.log10(x))
a[1][1].set_title('log')
plt.show()
```

Vishal Acharya

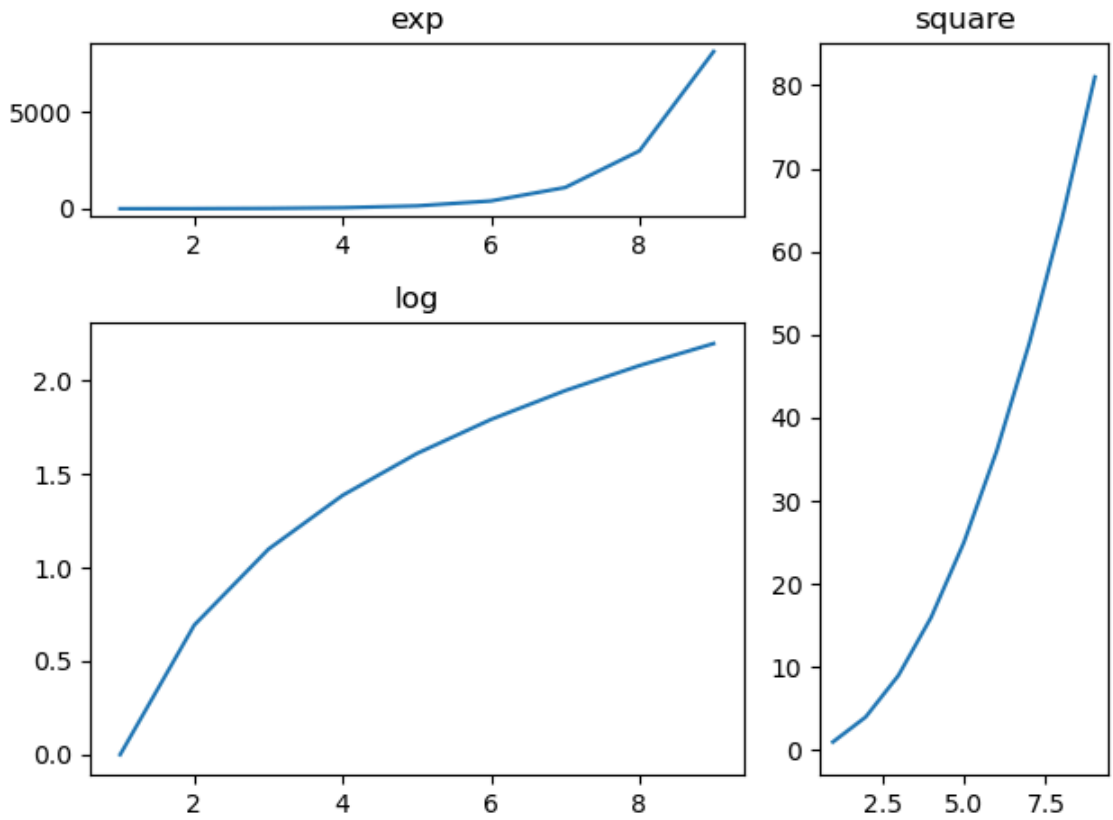


Matplotlib – Subplot2grid() function

- This function gives more flexibility in creating an axes object at a specific location of the grid. It also allows the axes object to be spanned across multiple rows or columns.
- In the following example, a 3X3 grid of the figure object is filled with axes objects of varying sizes in row and column spans, each showing a different plot.

In [42]:
```
import matplotlib.pyplot as plt
a1= plt.subplot2grid((3,3),(0,0),colspan=2)
a2=plt.subplot2grid((3,3),(0,2), rowspan=3)
a3=plt.subplot2grid((3,3),(1,0),rowspan=2, colspan=2)
import numpy as np
x=np.arange(1,10)
a2.plot(x, x*x)
a2.set_title('square')
a1.plot(x, np.exp(x))
a1.set_title('exp')
a3.plot(x, np.log(x))
a3.set_title('log')
plt.tight_layout()
plt.show()
```

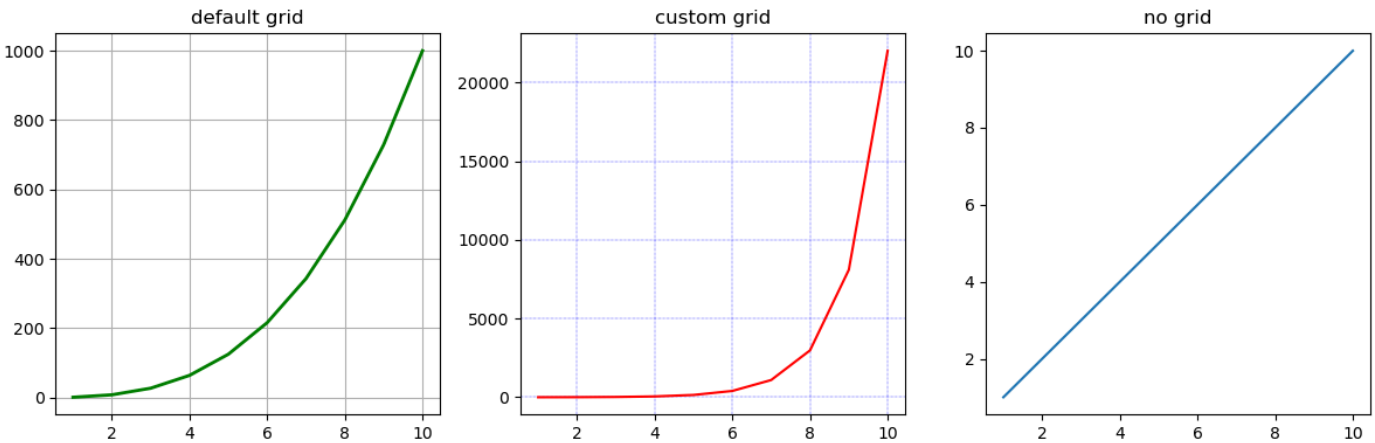
Vishal Acharya



Matplotlib – Grids

- The grid() function of axes object sets visibility of grid inside the figure to on or off. You can also display major / minor (or both) ticks of the grid. Additionally color, linestyle and linewidth properties can be set in the grid() function.

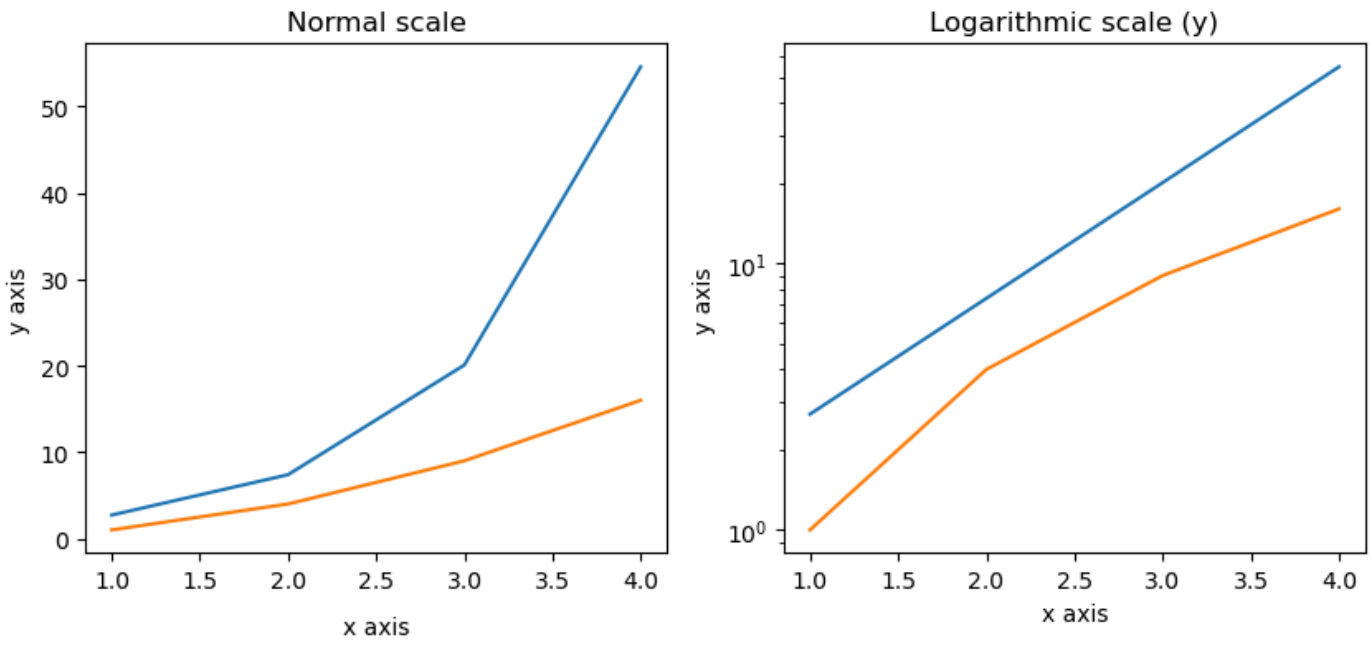
In [43]:
```
import matplotlib.pyplot as plt
import numpy as np
fig, axes = plt.subplots(1,3, figsize=(12,4))
x=np.arange(1,11)
axes[0].plot(x, x**3, 'g',lw=2)
axes[0].grid(True)
axes[0].set_title('default grid')
axes[1].plot(x, np.exp(x), 'r')
axes[1].grid(color='b', ls='-.', lw=0.25)
axes[1].set_title('custom grid')
axes[2].plot(x,x)
axes[2].set_title('no grid')
fig.tight_layout()
plt.show()
```



Vishal Acharya

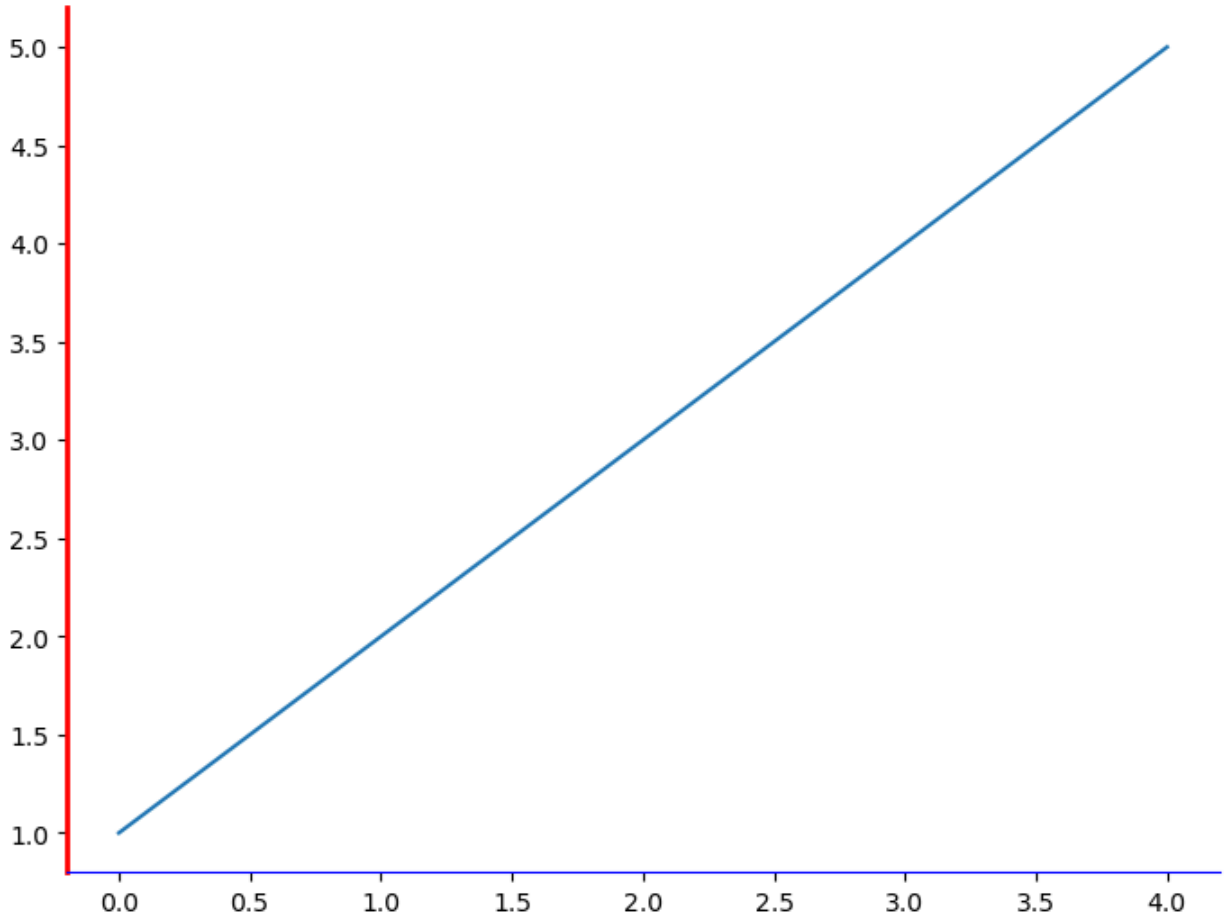
Matplotlib – Formatting Axes

In [44]:
```
import matplotlib.pyplot as plt
import numpy as np
fig, axes = plt.subplots(1, 2, figsize=(10,4))
x=np.arange(1,5)
axes[0].plot( x, np.exp(x))
axes[0].plot(x,x**2)
axes[0].set_title("Normal scale")
axes[1].plot (x, np.exp(x))
axes[1].plot(x, x**2)
axes[1].set_yscale("log")
axes[1].set_title("Logarithmic scale (y)")
axes[0].set_xlabel("x axis")
axes[0].set_ylabel("y axis")
axes[0].xaxis.labelpad = 10
axes[1].set_xlabel("x axis")
axes[1].set_ylabel("y axis")
plt.show()
```



In [45]:
```
import matplotlib.pyplot as plt
fig=plt.figure()
ax=fig.add_axes([0,0,1,1])
ax.spines['bottom'].set_color('blue')
ax.spines['left'].set_color('red')
ax.spines['left'].set_linewidth(2)
ax.spines['right'].set_color(None)
ax.spines['top'].set_color(None)
ax.plot([1,2,3,4,5])
plt.show()
```

Vishal Acharya

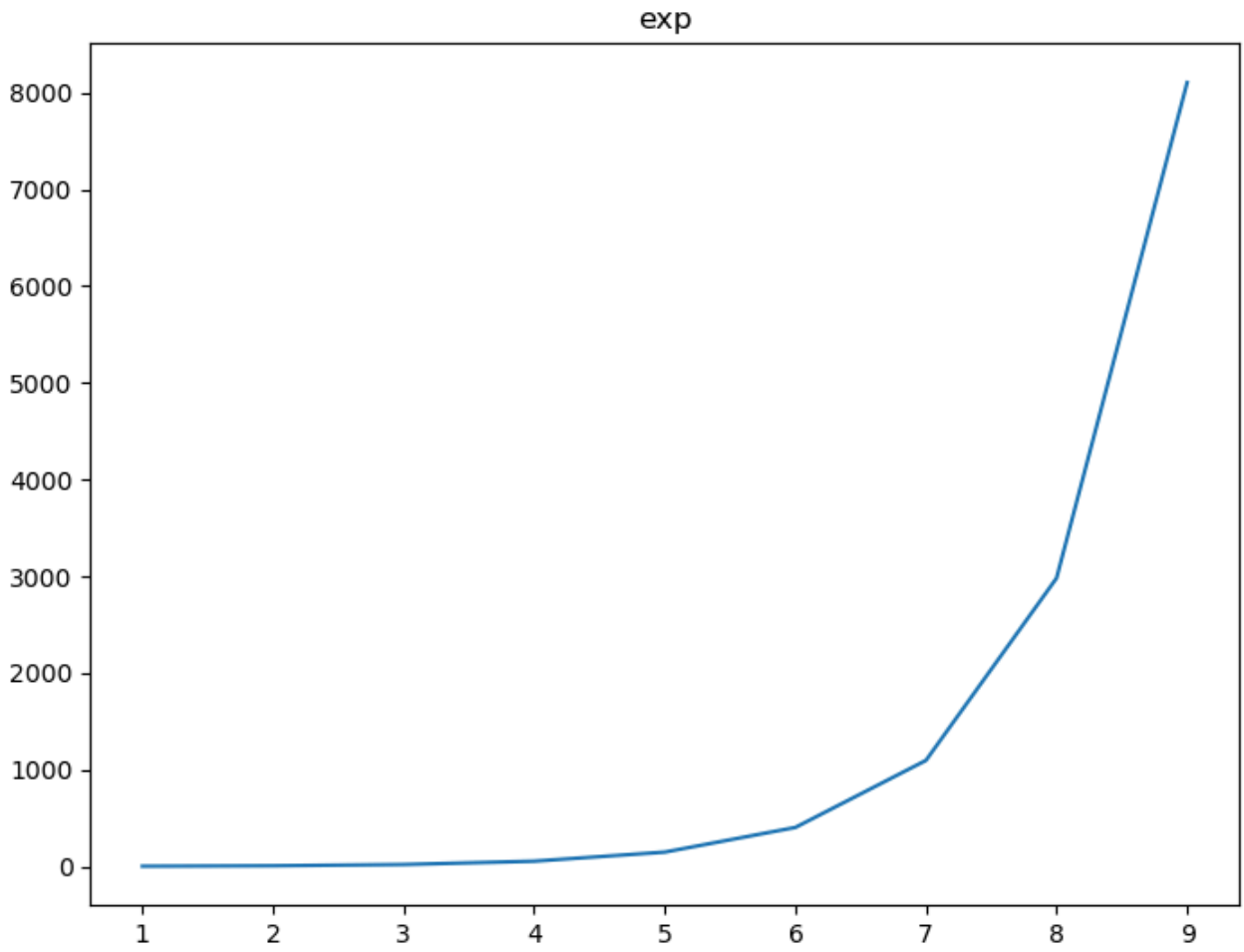


Matplotlib – Setting Limits

- Matplotlib automatically arrives at the minimum and maximum values of variables to be displayed along x, y (and z axis in case of 3D plot) axes of a plot. However, it is possible to set the limits explicitly by using set_xlim() and set_ylim() functions.

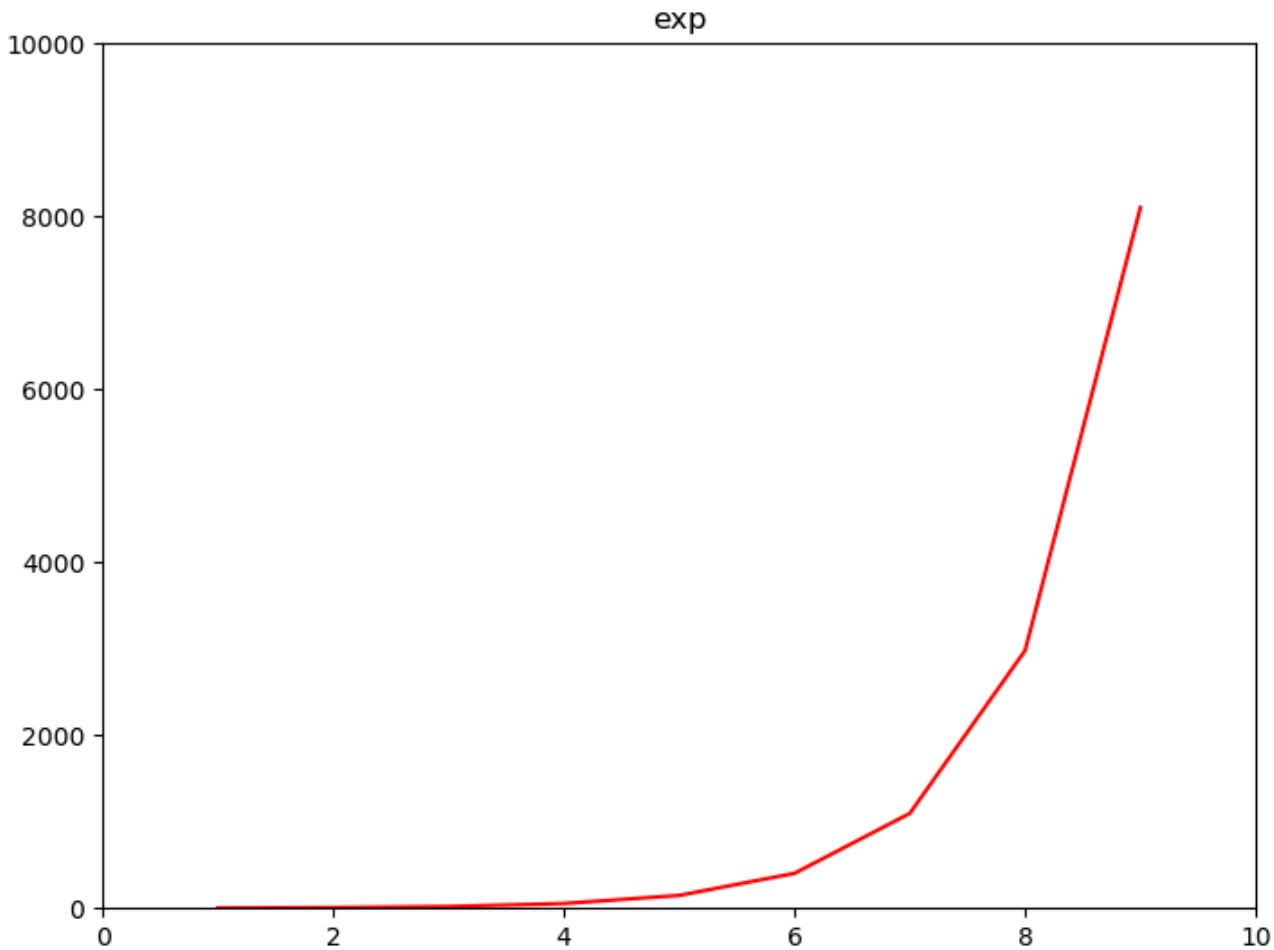
In [46]:
```
import matplotlib.pyplot as plt
fig=plt.figure()
a1=fig.add_axes([0,0,1,1])
import numpy as np
x=np.arange(1,10)
a1.plot(x, np.exp(x))
a1.set_title('exp')
plt.show()
```

Vishal Acharya



In [47]:
```
import matplotlib.pyplot as plt
fig=plt.figure()
a1=fig.add_axes([0,0,1,1])
import numpy as np
x=np.arange(1,10)
a1.plot(x, np.exp(x),'r')
a1.set_title('exp')
a1.set_ylim(0,10000)
a1.set_xlim(0,10)
plt.show()
```

Vishal Acharya

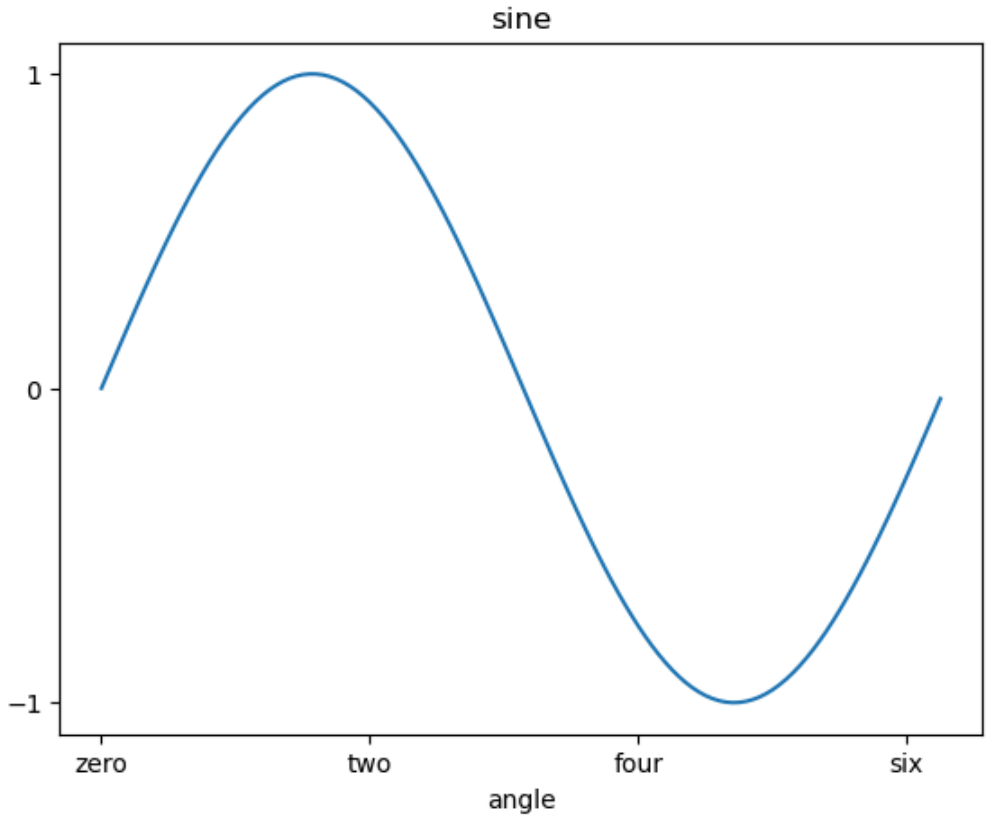


Matplotlib – Setting Ticks and Tick Labels

- Ticks are the markers denoting data points on axes. Matplotlib has so far - in all our previous examples - automatically taken over the task of spacing points on the axis.Matplotlib's default tick locators and formatters are designed to be generally sufficient in many common situations. Position and labels of ticks can be explicitly mentioned to suit specific requirements.
- The xticks() and yticks() function takes a list object as argument. The elements in the list denote the positions on corresponding action where ticks will be displayed.
- ax.set_xticks([2,4,6,8,10])
- Similarly, labels corresponding to tick marks can be set by set_xlabels() and set_ylabels() functions respectively.
- ax.set_xlabels(['two', 'four','six', 'eight', 'ten'])

In [50]:
```
import matplotlib.pyplot as plt
import numpy as np
import math
x=np.arange(0, math.pi*2, 0.05)
fig=plt.figure()
ax = fig.add_axes([0.1, 0.1, 0.8, 0.8]) # main axes
y=np.sin(x)
ax.plot(x, y)
ax.set_xlabel('angle')
ax.set_title('sine')
ax.set_xticks([0,2,4,6])
ax.set_xticklabels(['zero','two','four','six'])
ax.set_yticks([-1,0,1])
plt.show()
```

Vishal Acharya



bar chart

- A bar chart or bar graph is a chart or graph that presents categorical data with rectangular bars with heights or lengths proportional to the values that they represent. The bars can be plotted vertically or horizontally.
- A bar graph shows comparisons among discrete categories. One axis of the chart shows the specific categories being compared, and the other axis represents a measured value.

ax.bar(x, height, width, bottom, align)

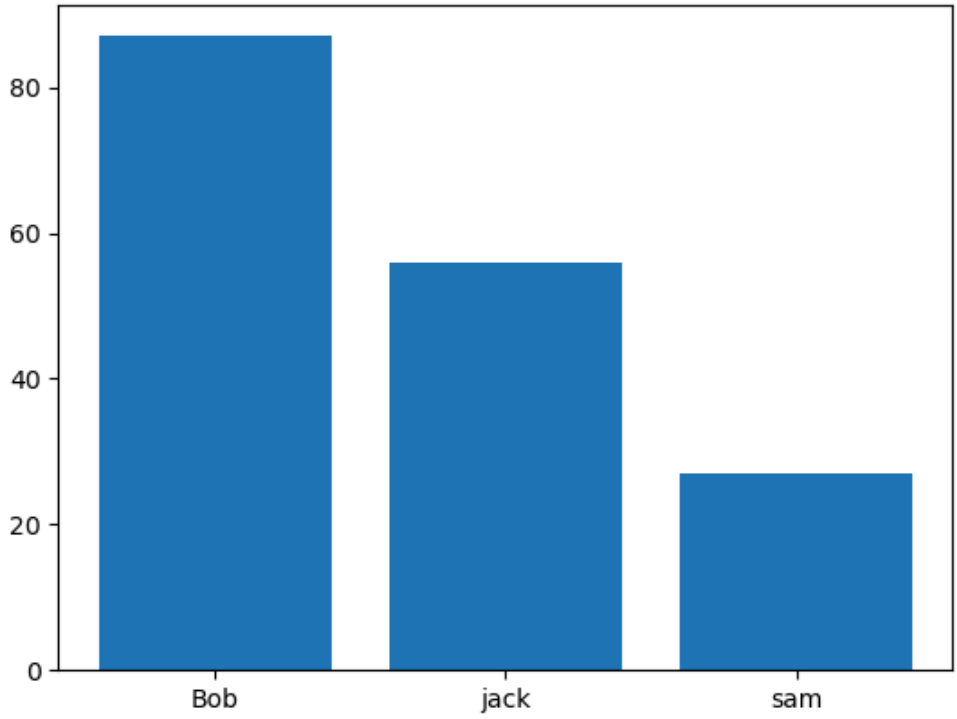
The function makes a bar plot with the bound rectangle of size (x −width=2; x + width=2; bottom; bottom + height).

The parameters to the function are:

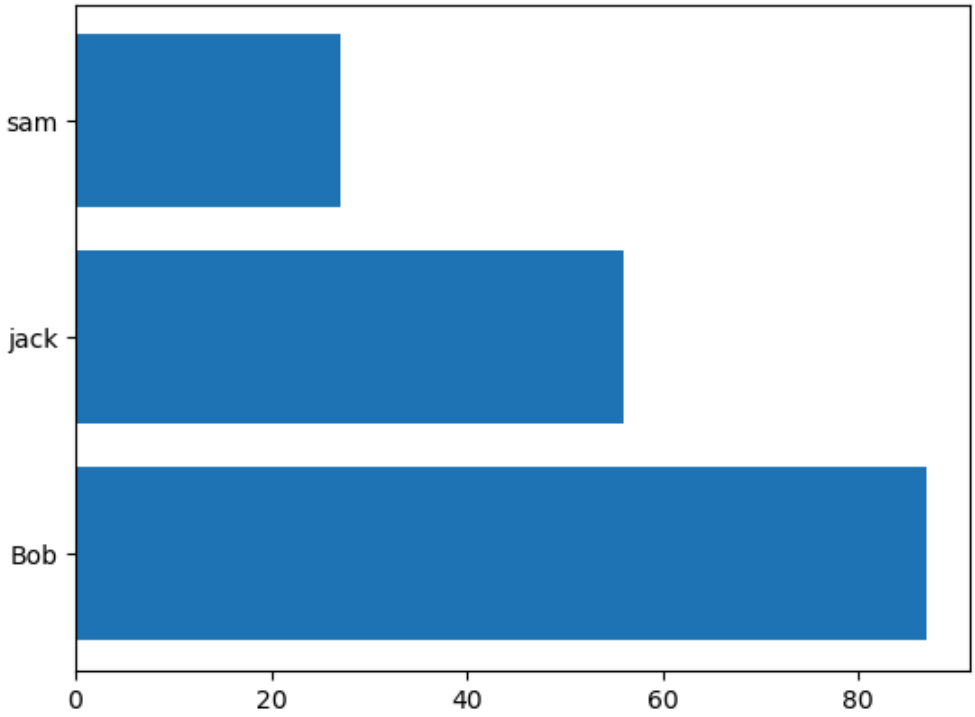
x	sequence of scalars representing the x coordinates of the bars. align controls if x is the bar center (default) or left edge.
height	scalar or sequence of scalars representing the height(s) of the bars
width	scalar or array-like, optional. the width(s) of the bars default 0.8
bottom	scalar or array-like, optional. the y coordinate(s) of the bars default None
align	{'center', 'edge'}, optional, default 'center'

In [74]:
```
student={"Bob":87,"jack":56,"sam":27}
names=list(student.keys())
values=list(student.values())
plt.bar(names,values)
plt.show()
```

Vishal Acharya

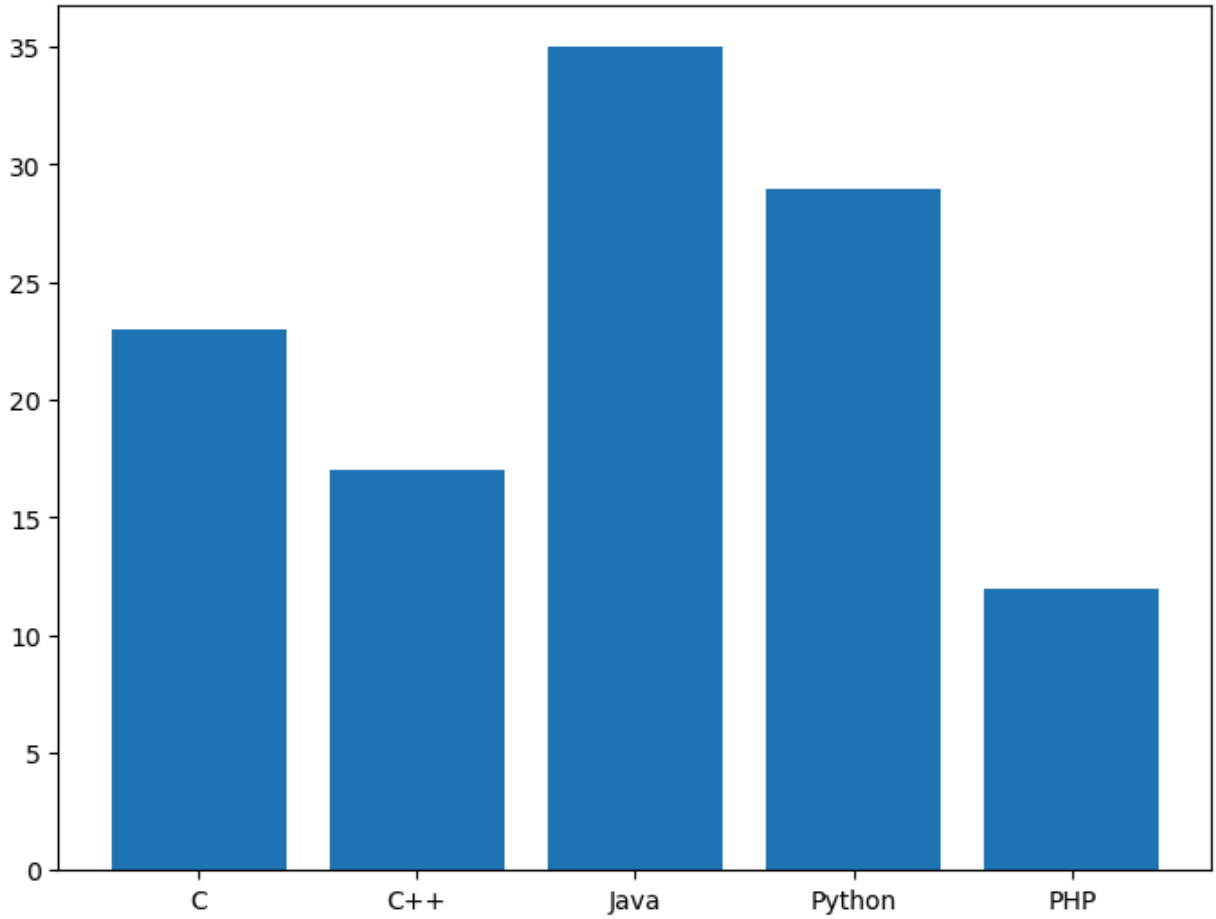


In [75]:
```
student={"Bob":87,"jack":56,"sam":27}
names=list(student.keys())
values=list(student.values())
plt.barh(names,values)
plt.show()
```



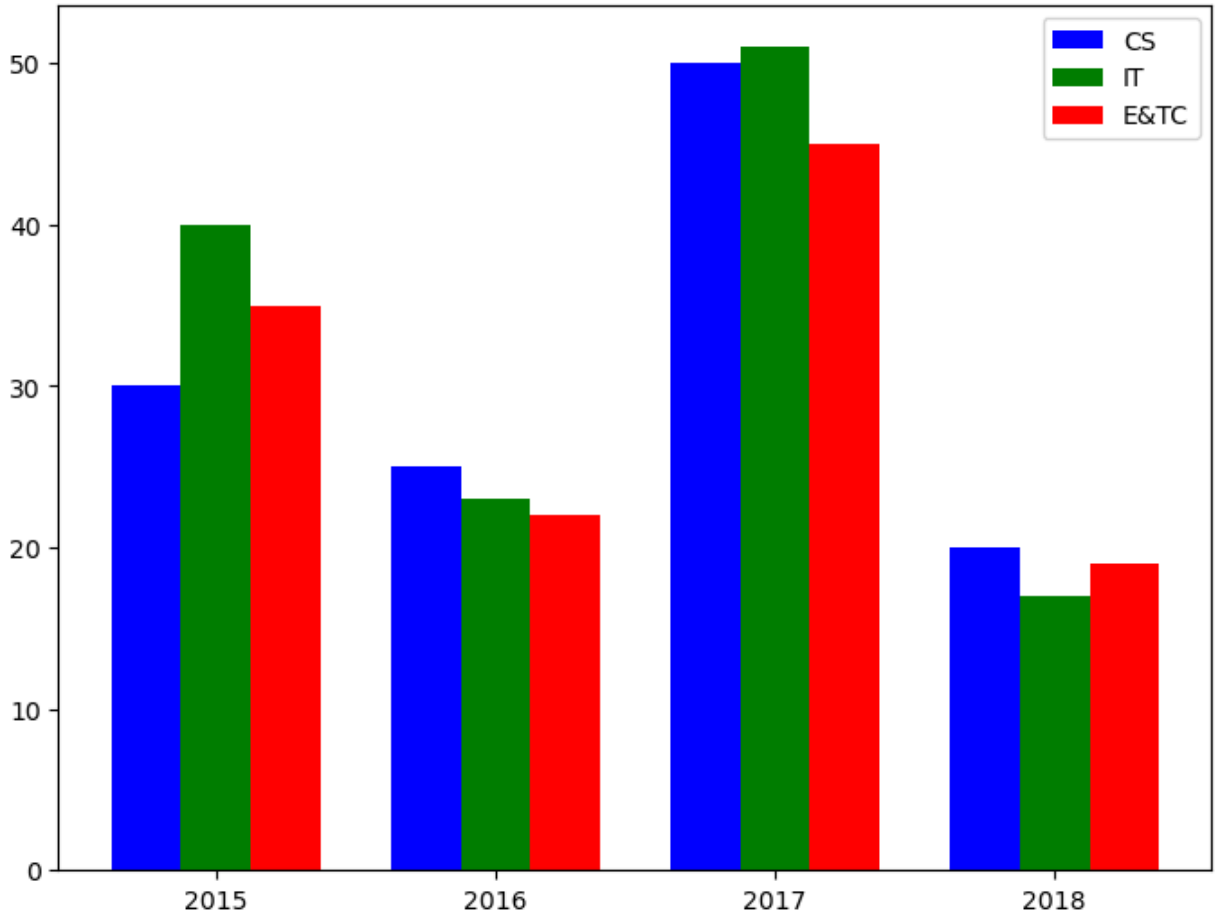
In [51]:
```
import matplotlib.pyplot as plt
fig=plt.figure()
ax=fig.add_axes([0,0,1,1])
langs=['C', 'C++', 'Java', 'Python', 'PHP']
students=[23,17,35,29,12]
ax.bar(langs,students)
plt.show()
```

Vishal Acharya



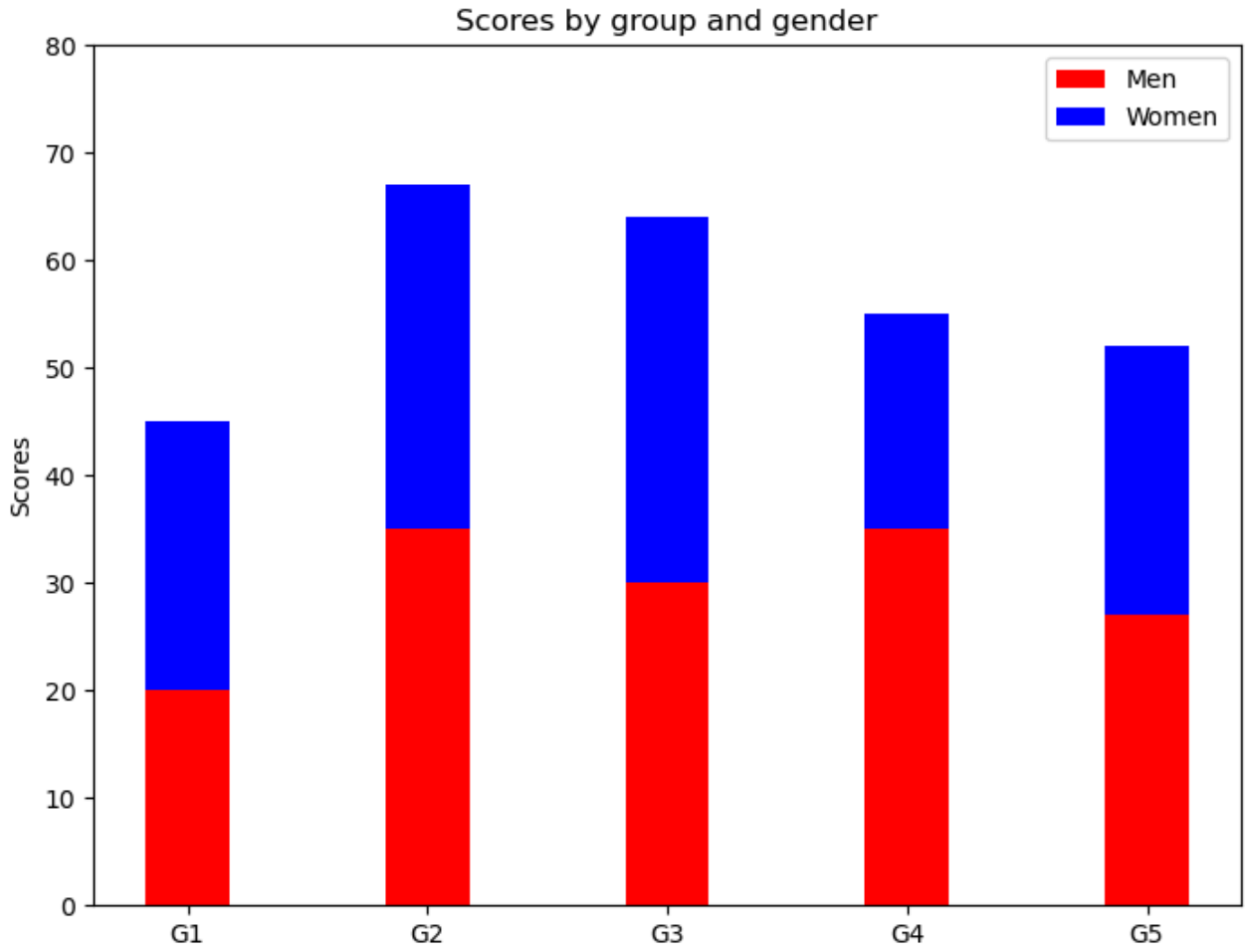
In [52]:
```
import numpy as np
import matplotlib.pyplot as plt
data = [[30, 25, 50, 20],
[40, 23, 51, 17],
[35, 22, 45, 19]]
X = np.arange(4)
fig=plt.figure()
ax=fig.add_axes([0,0,1,1])
ax.bar(X + 0.00, data[0], color = 'b', width = 0.25)
ax.bar(X + 0.25, data[1], color = 'g', width = 0.25)
ax.bar(X + 0.50, data[2], color = 'r', width = 0.25)
ax.set_xticks([0.25,1.25,2.25,3.25])
ax.set_xticklabels([2015,2016,2017,2018])
ax.legend(labels=['CS','IT','E&TC'])
plt.show()
```

Vishal Acharya



In [53]:
```
import numpy as np
import matplotlib.pyplot as plt
N = 5
menMeans = (20, 35, 30, 35, 27)
womenMeans = (25, 32, 34, 20, 25)
ind = np.arange(N) # the x locations for the groups
width = 0.35
fig=plt.figure()
ax=fig.add_axes([0,0,1,1])
ax.bar(ind, menMeans, width, color='r')
ax.bar(ind, womenMeans, width,bottom=menMeans, color='b')
ax.set_ylabel('Scores')
ax.set_title('Scores by group and gender')
ax.set_xticks(ind, ('G1', 'G2', 'G3', 'G4', 'G5'))
ax.set_yticks(np.arange(0, 81, 10))
ax.legend(labels=['Men', 'Women'])
plt.show()
```

Vishal Acharya



Matplotlib – Histogram

- A histogram is an accurate representation of the distribution of numerical data. It is an estimate of the probability distribution of a continuous variable. It is a kind of bar graph.
- To construct a histogram, follow these steps:
- Bin the range of values.
- Divide the entire range of values into a series of intervals.
- Count how many values fall into each interval.
- The bins are usually specified as consecutive, non-overlapping intervals of a variable.
- The matplotlib.pyplot.hist() function plots a histogram. It computes and draws the histogram of x.

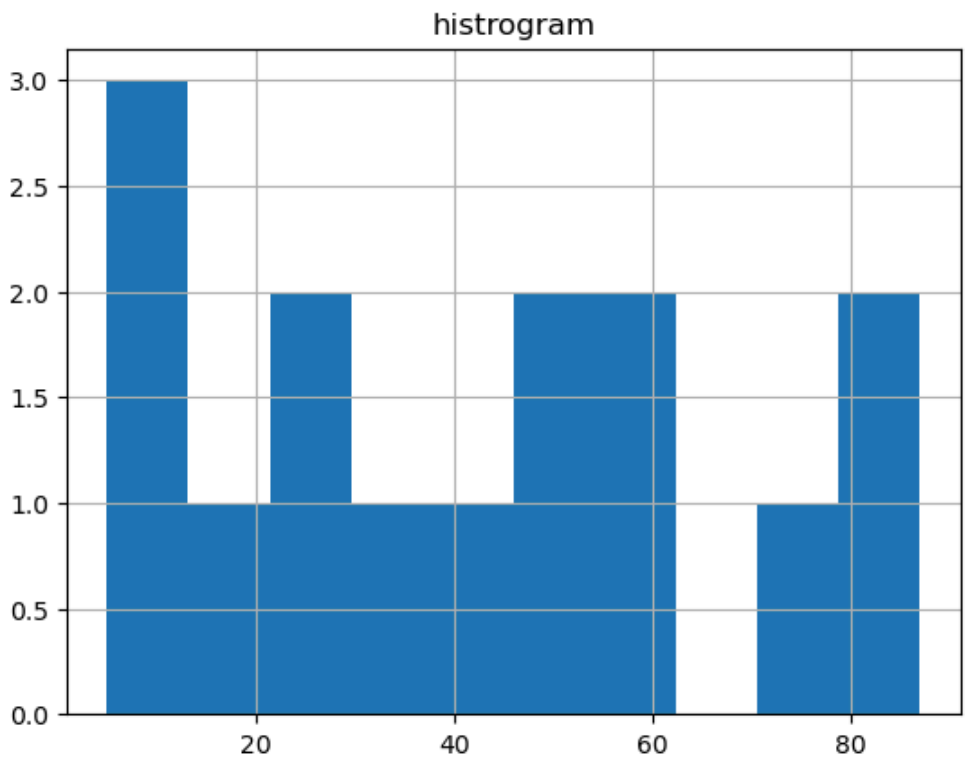
Vishal Acharya

Parameters

The following table lists down the parameters for a histogram:

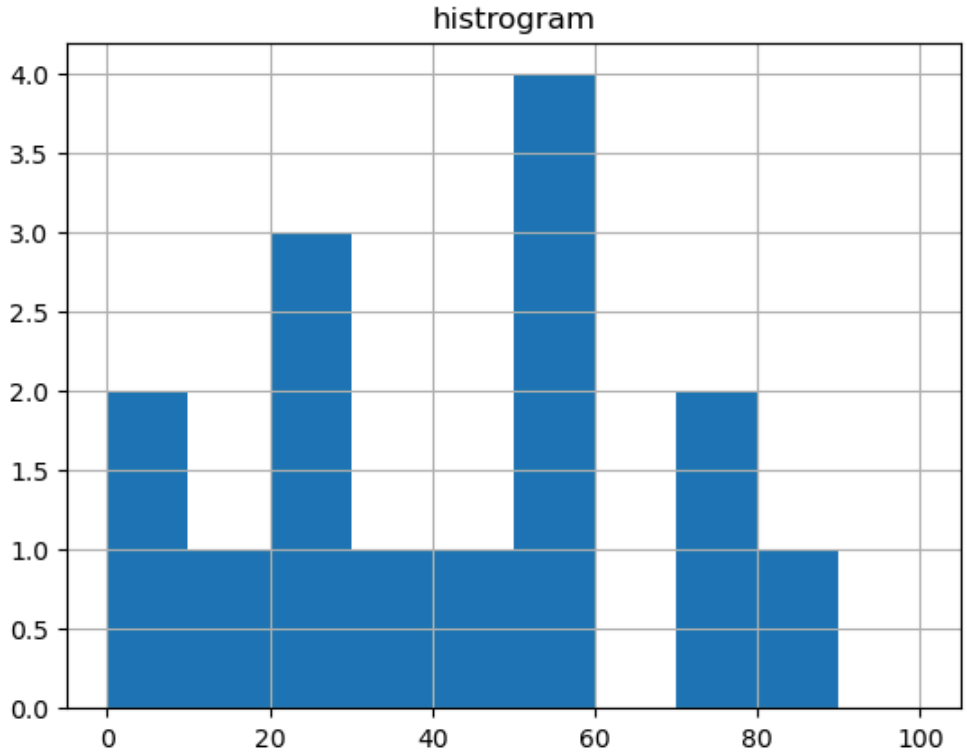
x	array or sequence of arrays
bins	integer or sequence or 'auto', optional
optional parameters	
range	The lower and upper range of the bins.
density	If True, the first element of the return tuple will be the counts normalized to form a probability density
cumulative	If True, then a histogram is computed where each bin gives the counts in that bin plus all bins for smaller values.
histtype	The type of histogram to draw. Default is 'bar' • 'bar' is a traditional bar-type histogram. If multiple data are given the bars are arranged side by side. • 'barstacked' is a bar-type histogram where multiple data are stacked on top of each other. • 'step' generates a lineplot that is by default unfilled. • 'stepfilled' generates a lineplot that is by default filled.

In [76]:
```
a = np.array([22,87,5,43,56,73,55,54,11,20,51,5,79,31,27])
plt.hist(a)
plt.title("histrogram")
plt.grid()
plt.show()
```

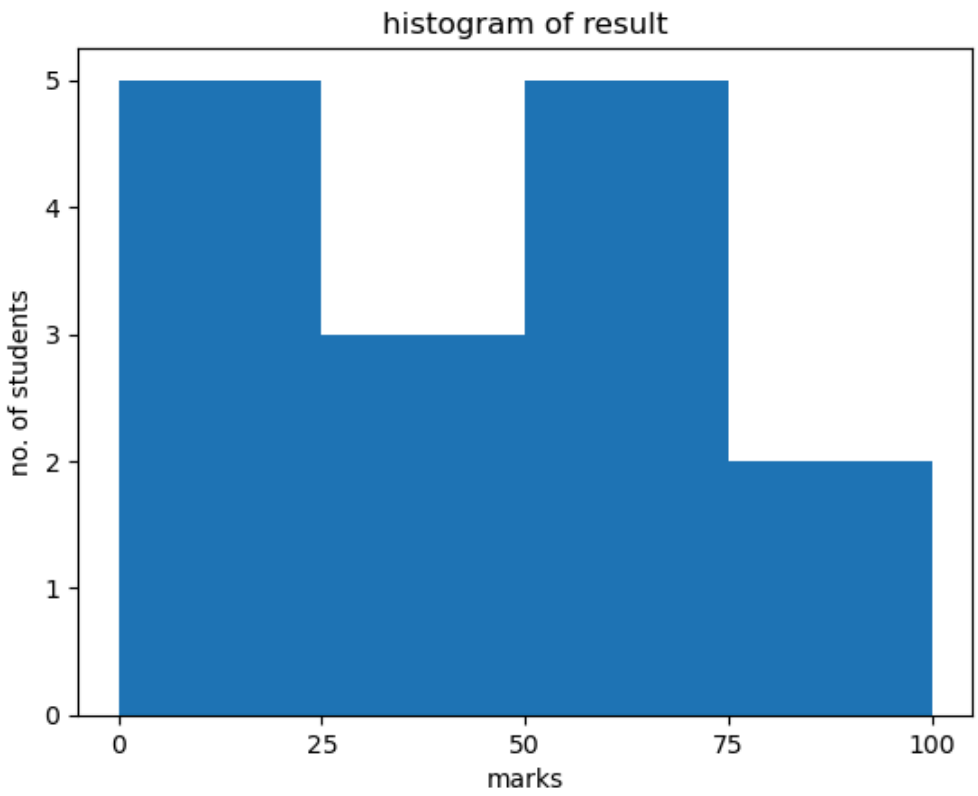


In [77]:
```
a = np.array([22,87,5,43,56,73,55,54,11,20,51,5,79,31,27])
plt.hist(a,bins=[0,10,20,30,40,50,60,70,80,90,100])
plt.title("histrogram")
plt.grid()
plt.show()
```

Vishal Acharya



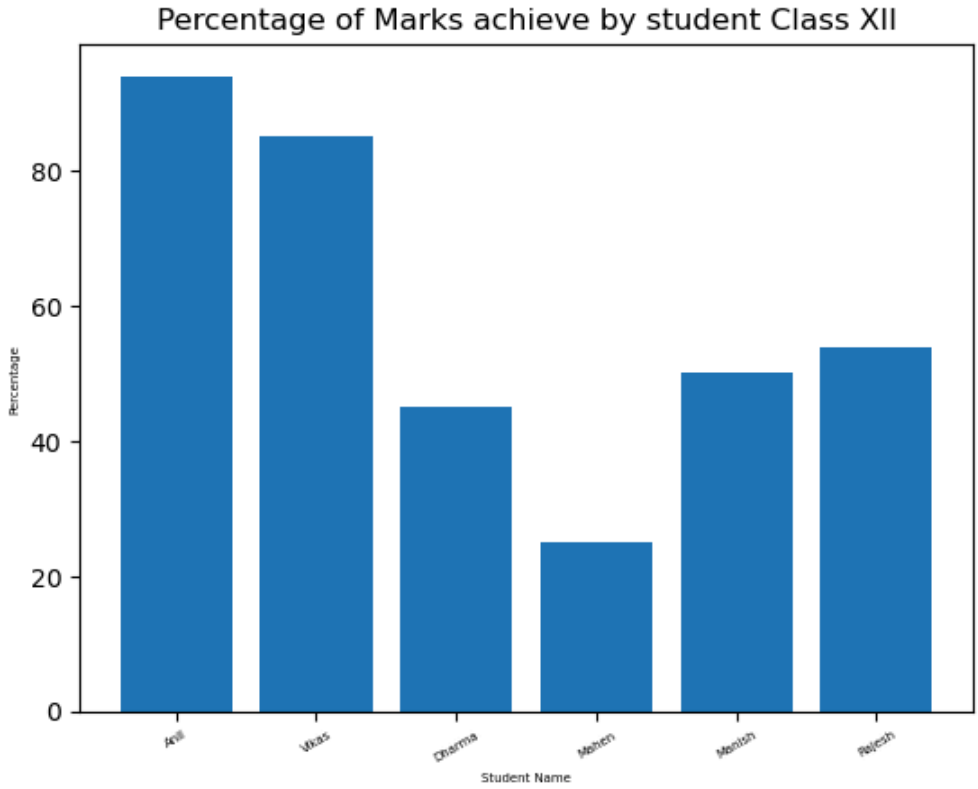
In [54]:
```
from matplotlib import pyplot as plt
import numpy as np
fig,ax=plt.subplots(1,1)
a = np.array([22,87,5,43,56,73,55,54,11,20,51,5,79,31,27])
ax.hist(a, bins = [0,25,50,75,100])
ax.set_title("histogram of result")
ax.set_xticks([0,25,50,75,100])
ax.set_xlabel('marks')
ax.set_ylabel('no. of students')
plt.show()
```



In [18]:
```
import matplotlib.pyplot as plt
import numpy as np
label = ['Anil', 'Vikas', 'Dharma',
'Mahen', 'Manish', 'Rajesh']
per = [94,85,45,25,50,54]
```

Vishal Acharya

```
index = np.arange(len(label))
plt.bar(index, per)
plt.xlabel('Student Name', fontsize=5)
plt.ylabel('Percentage', fontsize=5)
plt.xticks(index, label, fontsize=5,
rotation=30)
plt.title('Percentage of Marks achieve by student Class XII')
plt.show()
```



Matplotlib – Pie Chart

- A Pie Chart can only display one series of data. Pie charts show the size of items (called wedge) in one data series, proportional to the sum of the items. The data points in a pie chart are shown as a percentage of the whole pie.
- Matplotlib API has a pie() function that generates a pie diagram representing data in an array. The fractional area of each wedge is given by x/sum(x). If sum(x) < 1, then the values of x give the fractional area directly and the array will not be normalized. Theresulting pie will have an empty wedge of size 1 - sum(x).
- The pie chart looks best if the figure and axes are square, or the Axes aspect is equal.

Parameters

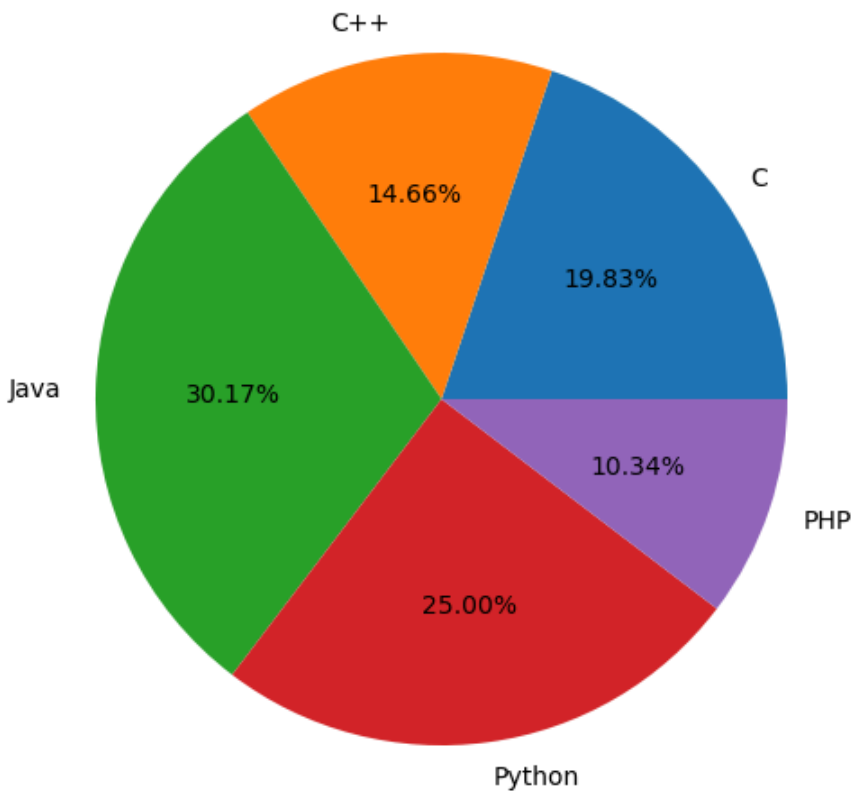
Following table lists down the parameters foe a pie chart:

x	array-like. The wedge sizes.
labels	list. A sequence of strings providing the labels for each wedge
Colors	A sequence of matplotlibcolorargs through which the pie chart will cycle. If None, will use the colors in the currently active cycle.
Autopct	string, used to label the wedges with their numeric value. The label will be placed inside the wedge. The format string will be fmt%pct.

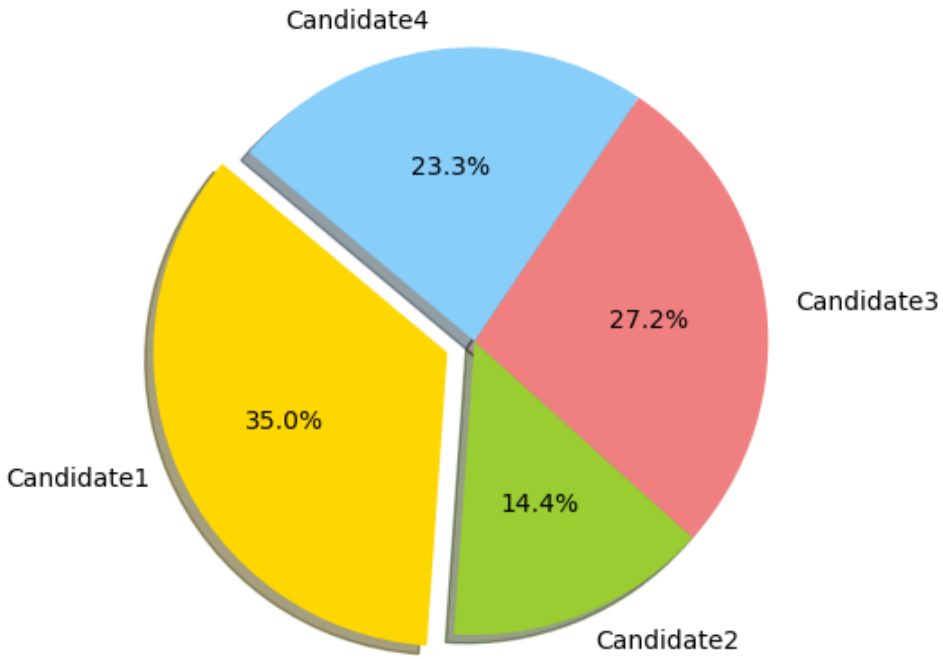
In [55]:
```
from matplotlib import pyplot as plt
import numpy as np
fig=plt.figure()
ax=fig.add_axes([0,0,1,1])
ax.axis('equal')
langs=['C', 'C++', 'Java', 'Python', 'PHP']
```

Vishal Acharya

```
students=[23,17,35,29,12]
ax.pie(students, labels=langs,autopct='%1.2f%%')
plt.show()
```



In [14]:
```
import matplotlib.pyplot as plt
# Data to plot
labels = 'Candidate1', 'Candidate2', 'Candidate3', 'Candidate4'
votes = [315, 130, 245, 210]
sizes=votes
colors = ['gold', 'yellowgreen', 'lightcoral', 'lightskyblue']
explode = (0.1, 0, 0, 0) # explode 1st slice
# Plot
plt.pie(sizes, explode=explode, labels=labels, colors=colors,
autopct='%1.1f%%', shadow=True, startangle=140)
plt.axis('equal')
plt.show()
```

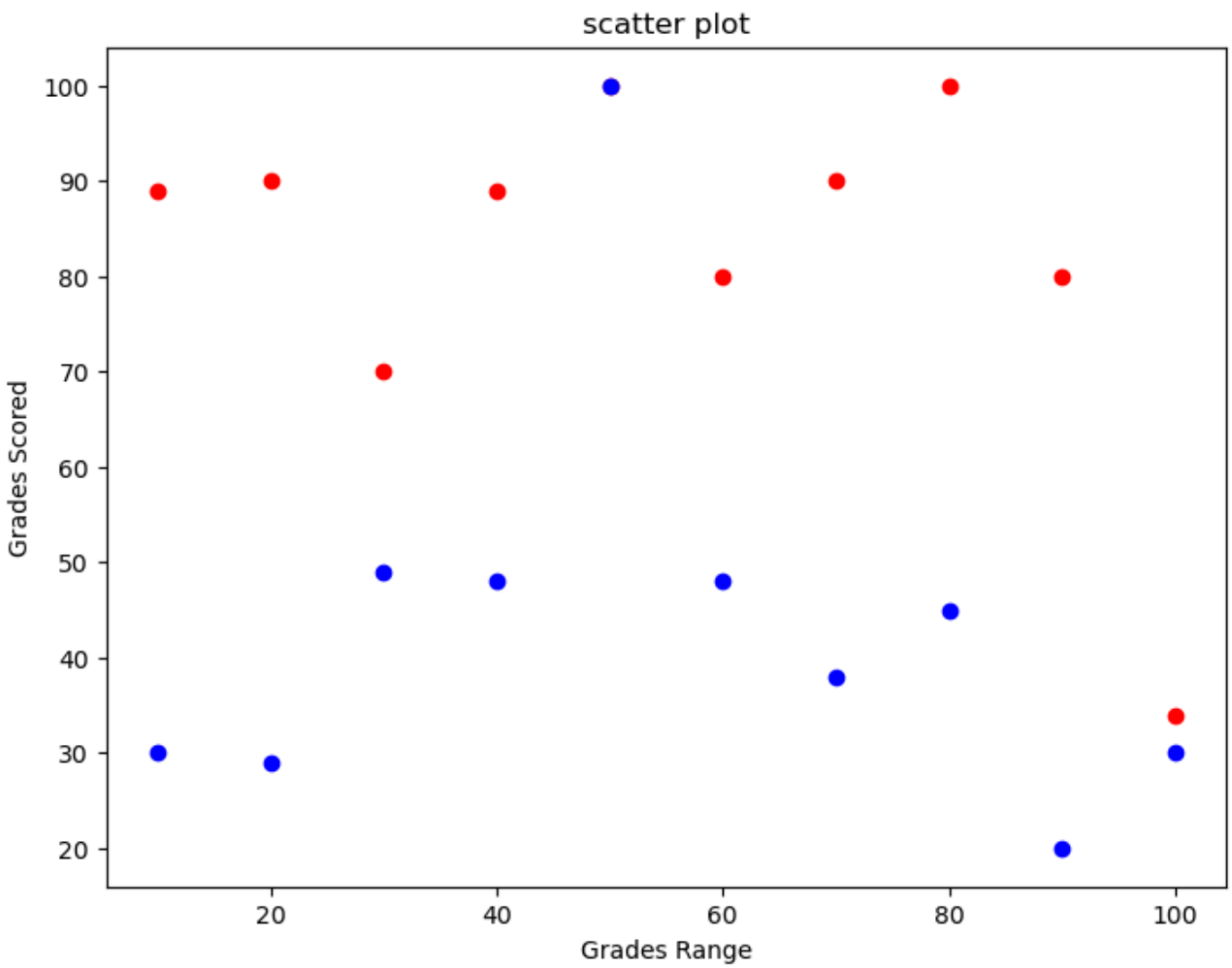


Vishal Acharya

Matplotlib – Scatter Plot

- Scatter plots are used to plot data points on horizontal and vertical axis in the attempt to show how much one variable is affected by another. Each row in the data table is represented by a marker the position depends on its values in the columns set on the X and Y axes. A third variable can be set to correspond to the color or size of the markers, thus adding yet another dimension to the plot.
- The script below plots a scatter diagram of grades range vs grades of boys and girls in two different colors.

In [57]:
```
import matplotlib.pyplot as plt
girls_grades = [89, 90, 70, 89, 100, 80, 90, 100, 80, 34]
boys_grades = [30, 29, 49, 48, 100, 48, 38, 45, 20, 30]
grades_range = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
fig=plt.figure()
ax=fig.add_axes([0,0,1,1])
ax.scatter(grades_range, girls_grades, color='r')
ax.scatter(grades_range, boys_grades, color='b')
ax.set_xlabel('Grades Range')
ax.set_ylabel('Grades Scored')
ax.set_title('scatter plot')
plt.show()
```

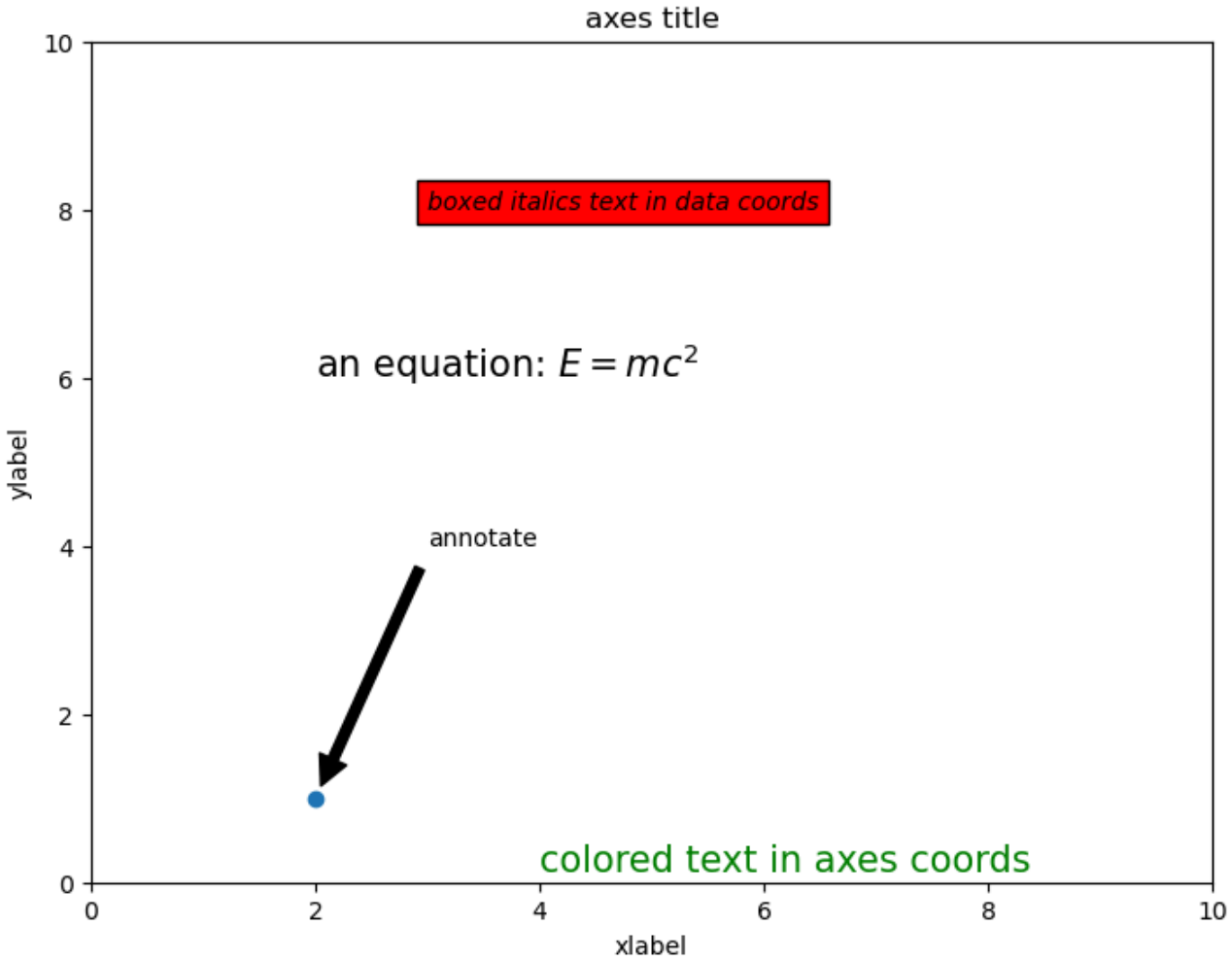


Vishal Acharya

Matplotlib – Working With Text

text	Add text at an arbitrary location of the Axes.
annotate	Add an annotation, with an optional arrow, at an arbitrary location of theAxes.
xlabel	Add a label to the Axes's x-axis.
ylabel	Add a label to the Axes's y-axis.
title	Add a title to the Axes.
figtext	Add text at an arbitrary location of the Figure.
suptitle	Add a title to the Figure.

In [58]:
```
import matplotlib.pyplot as plt
fig = plt.figure()
ax = fig.add_axes([0,0,1,1])
ax.set_title('axes title')
ax.set_xlabel('xlabel')
ax.set_ylabel('ylabel')
ax.text(3, 8, 'boxed italics text in data coords', style='italic', bbox={'facecolor': 'red'})
ax.text(2, 6, r'an equation: $E=mc^2$', fontsize=15)
ax.text(4, 0.05, 'colored text in axes coords',
verticalalignment='bottom', color='green', fontsize=15)
ax.plot([2], [1], 'o')
ax.annotate('annotate', xy=(2, 1), xytext=(3, 4),
arrowprops=dict(facecolor='black', shrink=0.05))
ax.axis([0, 10, 0, 10])
plt.show()
```



Vishal Acharya

Image Functions

Function	Description
Imread	Read an image from a file into an array.
Imsave	Save an array as in image file.
Imshow	Display an image on the axes.

Axis Functions

Function	Description
Axes	Add axes to the figure.
Text	Add text to the axes.
Title	Set a title of the current axes.
Xlabel	Set the x axis label of the current axis.
Xlim	Get or set the x limits of the current axes.
Xscale	Set the scaling of the x-axis.
Xticks	Get or set the x-limits of the current tick locations and labels.
Ylabel	Set the y axis label of the current axis.
Ylim	Get or set the y-limits of the current axes.
Yscale	Set the scaling of the y-axis.
Yticks	Get or set the y-limits of the current tick locations and labels.

Figure Functions

Function	Description
Figtext	Add text to figure.
Figure	Creates a new figure.
Show	Display a figure.
Savefig	Save the current figure.
Close	Close a figure window.

AREA PLOT

What is an Area Plot?

Vishal Acharya

An area plot displays quantitative data visually by filling the area under a line graph. It is useful for:

Showing trends over time

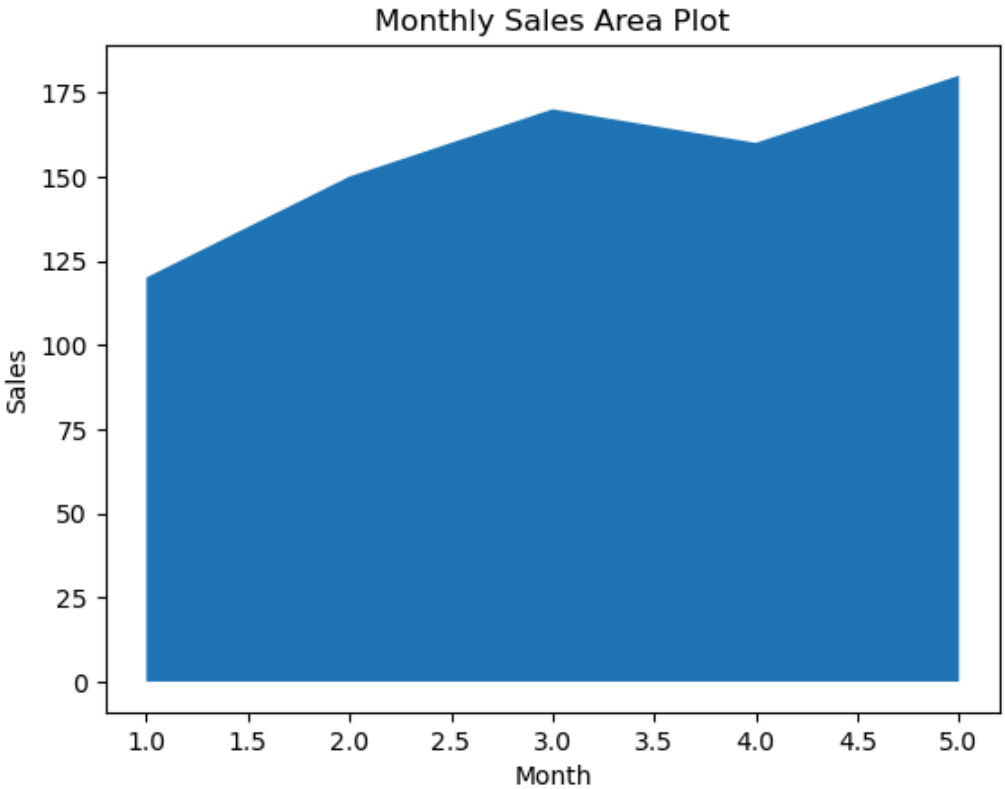
Comparing magnitudes

Visualizing cumulative data

plt.fill_between(x, y)

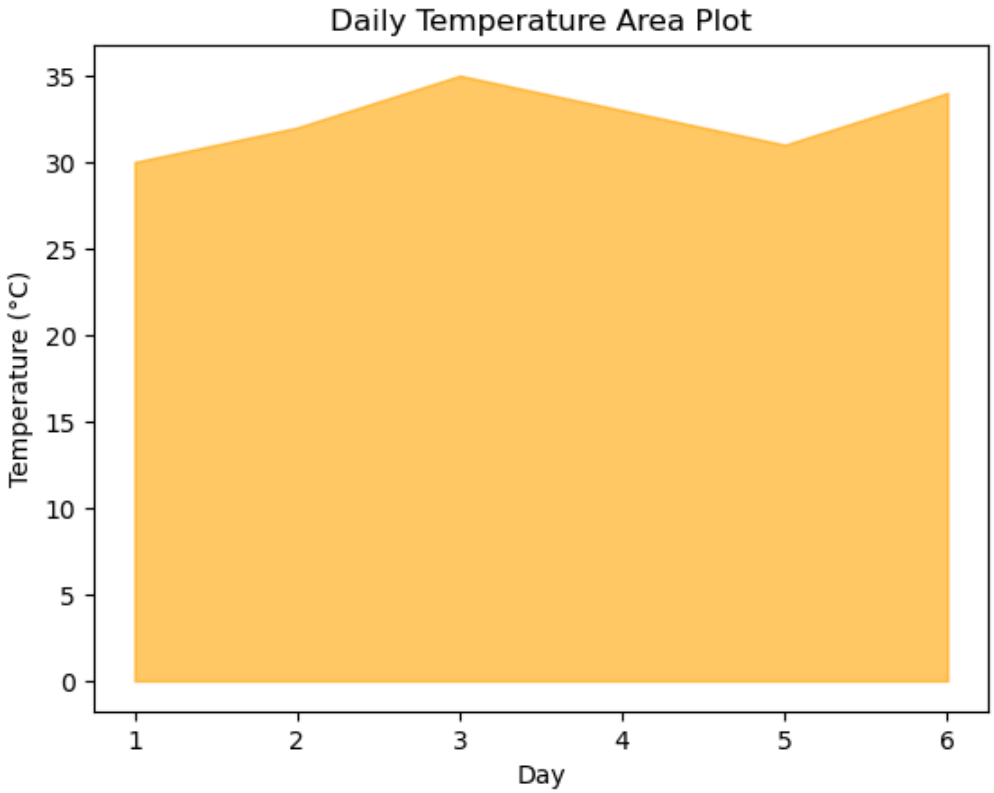
plt.stackplot(x, y1, y2, y3)

In [1]:
```
import matplotlib.pyplot as plt

months = [1, 2, 3, 4, 5]
sales = [120, 150, 170, 160, 180]

plt.fill_between(months, sales)
plt.xlabel("Month")
plt.ylabel("Sales")
plt.title("Monthly Sales Area Plot")

plt.show()
```



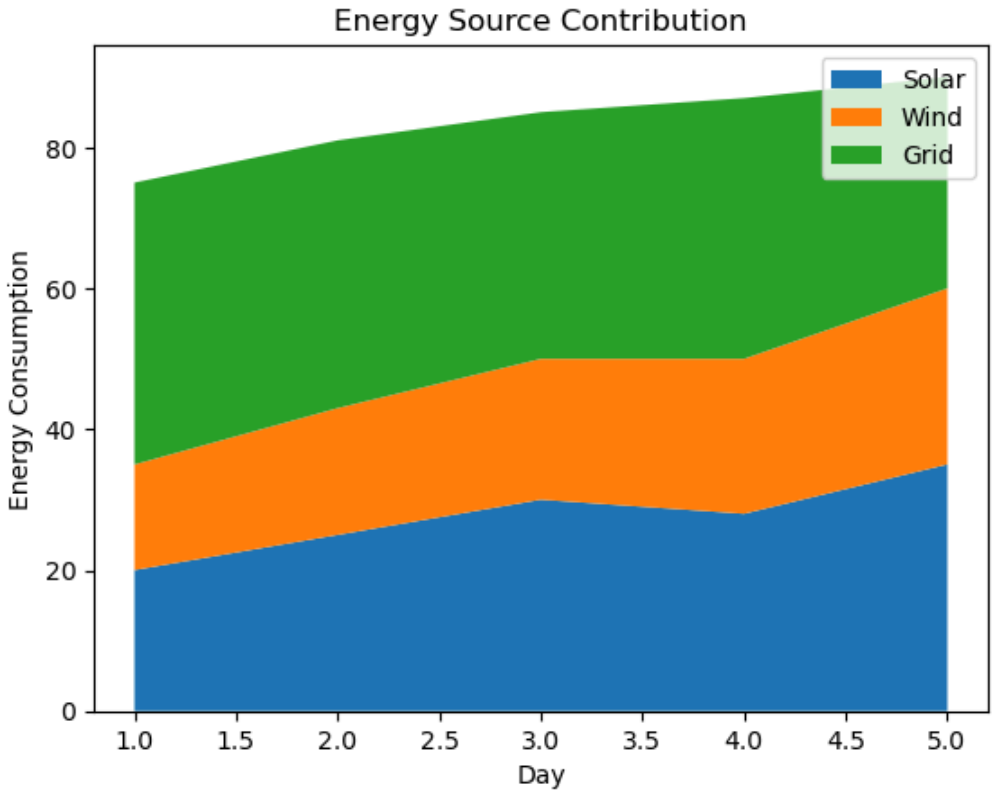
In [2]:
```
import matplotlib.pyplot as plt

days = [1, 2, 3, 4, 5, 6]
temperature = [30, 32, 35, 33, 31, 34]

plt.fill_between(days, temperature, color="orange", alpha=0.6)
plt.xlabel("Day")
plt.ylabel("Temperature (°C)")
plt.title("Daily Temperature Area Plot")

plt.show()
```

Vishal Acharya



In [3]:
```
import matplotlib.pyplot as plt

days = [1, 2, 3, 4, 5]

solar = [20, 25, 30, 28, 35]
wind = [15, 18, 20, 22, 25]
grid = [40, 38, 35, 37, 30]

plt.stackplot(days, solar, wind, grid,
              labels=["Solar", "Wind", "Grid"])

plt.xlabel("Day")
plt.ylabel("Energy Consumption")
plt.title("Energy Source Contribution")
plt.legend()

plt.show()
```

Vishal Acharya



Purpose of where The where parameter allows you to fill only a specific region of the graph based on a condition.

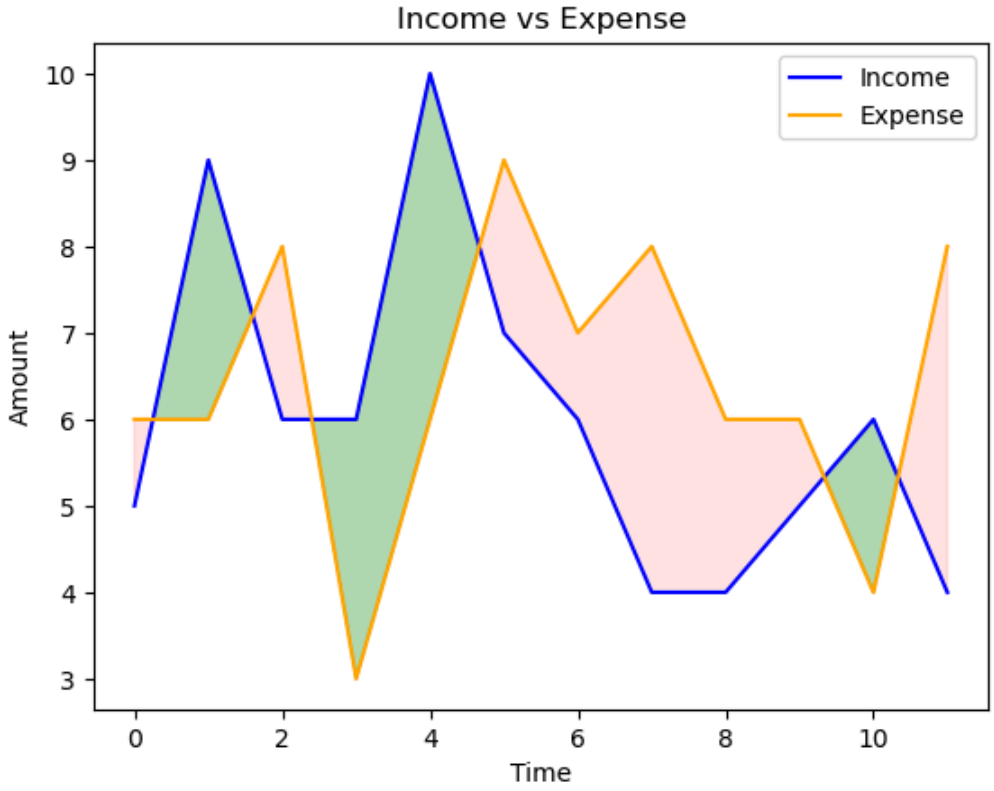
Basic syntax: plt.fill_between(x, y, where=condition, alpha=value)

condition → Boolean list/array (True / False)

alpha → Transparency (0 to 1)

In [4]:
```
import numpy as np
import matplotlib.pyplot as plt

# Define the numpy arrays
Time = np.arange(12)
income = np.array([5, 9, 6, 6, 10, 7, 6, 4, 4, 5, 6, 4])
expense = np.array([6, 6, 8, 3, 6, 9, 7, 8, 6, 6, 4, 8])

# Plot the lines for income and expense
plt.plot(Time, income, label='Income', color='blue')
plt.plot(Time, expense, label='Expense', color='orange')

# Fill the area between the lines with different colors
plt.fill_between(Time, income, expense, where=(income >= expense), interpolate=True, color='green', alpha=
plt.fill_between(Time, income, expense, where=(income < expense), interpolate=True, color='red', alpha=0.1

# Add labels and legend
plt.xlabel('Time')
plt.ylabel('Amount')
plt.title('Income vs Expense')
plt.legend()

# Show the plot
plt.show()
```

Vishal Acharya



In[ ]:

Vishal Acharya